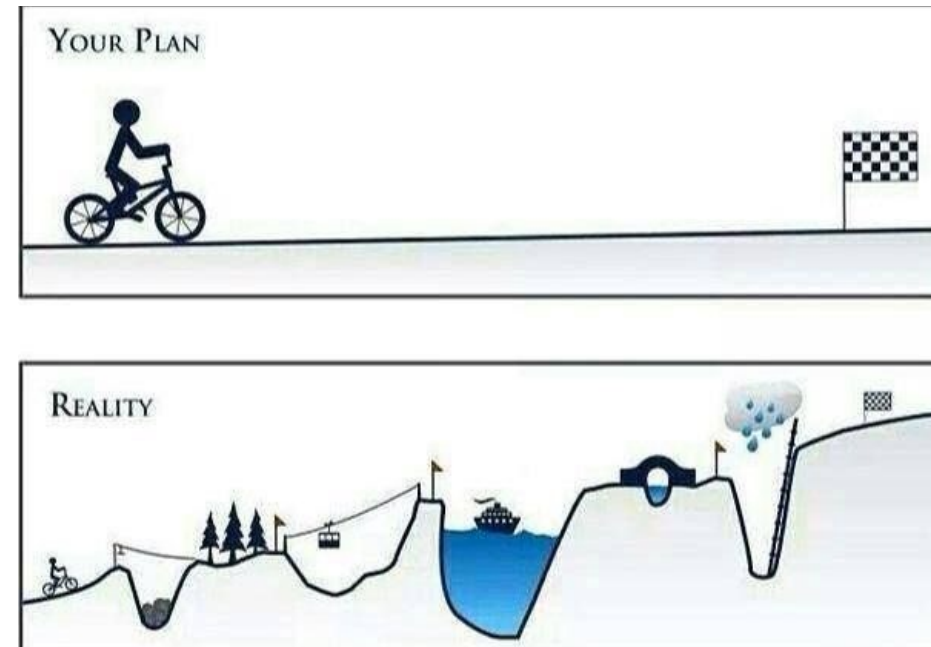


# Planning

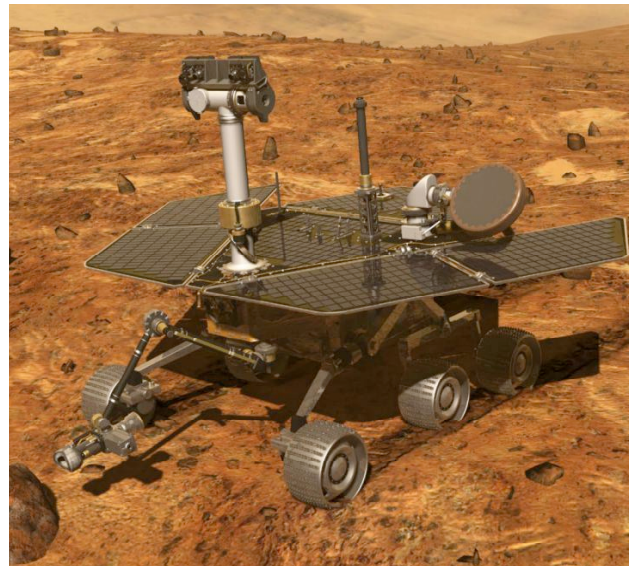
---



# Application

---

- ❑ Activity Planning for the Mars Exploration Rovers
- ❑ Question – How much time on an average does it take for a signal from Earth to reach Mars?



# Planning

---

□ Given a logical description of

- The world states
- Possible actions
- Initial state
- goal conditions

□ Find a sequence of actions (a plan of actions) that transforms the initial state to a state where the goal conditions are satisfied.

# Planning vs Problem Solving

---

- ❑ Often solve similar problems

- ❑ Primary difference – explicit and logic-based representation

- States/Situations – logical description of the world that allows the agent to reason about
- Goal Conditions – logical sentences vs goal test
- Operators/Actions – transformations on logical sentences that allows the agent to reason about the effects of actions
  - Facilitates divide and conquer strategy – solve subgoals independently

# Assumptions of the Problem World

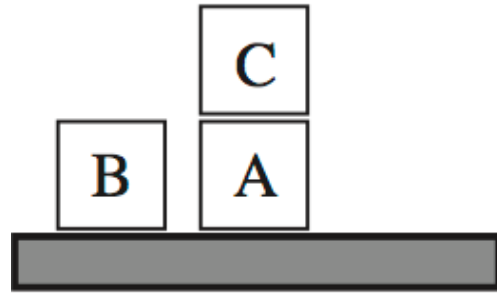
---

- ❑ Atomic Actions – Actions are indivisible
- ❑ No concurrent actions are allowed
- ❑ Deterministic – The results of the actions are completely determined. (no uncertainty)
- ❑ Single Agent
- ❑ Omniscient Agent (no sensing is necessary)
- ❑ Closed World Assumption – Everything known to be true in the world is included in the state description. Anything that is not listed is false.

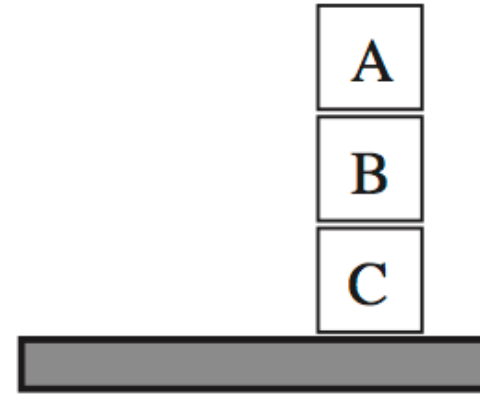
# Example: Blocks World

---

□ Transform the initial arrangement of blocks to the goal state



Start State



Goal State

□ Constraints

- Only one block can be on another block
- Any number of blocks can be on the table
- The hand can only hold one block

# Specifying a Planning Problem

---

## □ Planning Domain Definition Language (PDDL, STRIPS)

- Initial State
- Actions
- Results
- Goal test

# PDDL - State

---

## □ Conjunction of ground functionless atoms

### ○ Positive ground literals

- $At(Robot1, Room1) \wedge At(Robot2, Room2)$
- $At(Home) \wedge Have(Milk) \wedge Have(Bananas) \wedge \dots$
- $On(A, Table) \wedge On(B, Table) \wedge On(C, A) \wedge Block(A) \wedge Block(B) \wedge Block(C) \wedge Clear(B) \wedge Clear(C)$

### ○ The following are not valid representations of a state

- $\neg At(Home)$  – a negative literal
- $At(x, y)$  – not ground
- $At(Left\_Wheel(Robot1), Room1)$  – uses a function symbol

### ○ Close-world assumption

- If  $At(Home)$  is not present, assume  $\neg At(Home)$



# PDDL - Goal Test

---

## □ Goal

- Conjunction of literals (positive or negative, possibly with variables)
- Variables are existentially quantified
- A partially specified state
  - $On(z, y) \wedge On(y, x)$
  - $At(Home) \wedge Have(Milk) \wedge Have(Bananas)$
  - $At(x) \wedge Sells(x, Milk) \wedge Supermarket(x)$

## □ A state $s$ satisfies goal $g$ if $s$ contains (unifies with) all the literals of $g$

- $On(C, B) \wedge On(B, A)$
- $At(Home) \wedge Have(Milk) \wedge Have(Bananas) \wedge Rich \wedge Famous$
- $At(BigBazar) \wedge Sells(BigBazar, Milk) \wedge Supermarket(BigBazar)$

# PDDL - Actions

---

- ❑ Actions are modeled as state transformations
- ❑ Defined by a set of action schemas that implicitly define *Actions(s)* and *Results(s, a)*
- ❑ Description of an action should only mention the changes

*Action(Fly(p, from, to),*  
*PRECOND: At(p, from)  $\wedge$  Plane(p)  $\wedge$  Airport(from)  $\wedge$  Airport(to)*  
*EFFECT:  $\neg$ At(p, from)  $\wedge$  At(p, to))*

# PDDL – Actions (2)

---

- ❑ Precondition – What must be true for the action to be applicable
- ❑ Effect – Changes to the state as a result of taking the action
- ❑ Conjunction of literals (positive or negative)

*Action(Fly(p, from, to),*  
*PRECOND: At(p, from)  $\wedge$  Plane(p)  $\wedge$  Airport(from)  $\wedge$  Airport(to)*  
*EFFECT:  $\neg$ At(p, from)  $\wedge$  At(p, to))*

*Action(Fly(P1, CJB, DEL),* (ground action)  
*PRECOND: At(P1, CJB)  $\wedge$  Plane(P1)  $\wedge$  Airport(CJB)  $\wedge$  Airport(DEL)*  
*EFFECT:  $\neg$ At(P1, CJB)  $\wedge$  At(P1, DEL))*

# PDDL – Actions (3)

---

- ❑ Actions  $a$  can be executed in state  $s$  if  $s$  entails the precondition of  $a$

$$(a \in ACTIONS(s)) \Leftrightarrow s \models PRECOND(a)$$

- ❑ Where all variables in  $a$  are universally quantified
- ❑ For example

$Action(Fly(p, from, to),$   
 $PRECOND: At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$   
 $EFFECT: \neg At(p, from) \wedge At(p, to))$

$\forall p, from, to (Fly(p, from, to) \in ACTIONS(s)) \Leftrightarrow s \models (At(p, from) \wedge$   
 $Plane(p) \wedge Airport(from) \wedge Airport(to))$

# PDDL – Actions(4)

---

- ❑ The result of executing action  $a$  in state  $s$  is defined as a state  $s'$ 
  - State  $s'$  contains the literals of  $s$  minus negative literals in  $EFFECT$  plus positive literals in  $EFFECT$
- ❑ Negated literals in  $EFFECT$  called delete list  $DEL(a)$
- ❑ Positive literals in  $EFFECT$  called add list  $ADD(a)$
- ❑  $RESULT(s, a) = (s - DEL(a)) \cup ADD(a)$

# PDDL – Actions (5)

---

- ❑ For action schemas, any variable in *EFFECT* must also appear in *PRECOND*
  - *RESULT(s, a)* will therefore have only ground atoms
- ❑ Time is implicit in action schemas
  - Precondition refers to time  $t$
  - Effect refers to time  $t+1$

# PDDL – Example Air Cargo Transport

---

$Init(At(C_1, SFO) \wedge At(C_2, JFK) \wedge At(P_1, SFO) \wedge At(P_2, JFK)$   
     $\wedge Cargo(C_1) \wedge Cargo(C_2) \wedge Plane(P_1) \wedge Plane(P_2)$   
     $\wedge Airport(JFK) \wedge Airport(SFO))$   
 $Goal(At(C_1, JFK) \wedge At(C_2, SFO))$   
 $Action(Load(c, p, a),$   
    PRECOND:  $At(c, a) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$   
    EFFECT:  $\neg At(c, a) \wedge In(c, p)$ )  
 $Action(Unload(c, p, a),$   
    PRECOND:  $In(c, p) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$   
    EFFECT:  $At(c, a) \wedge \neg In(c, p)$ )  
 $Action(Fly(p, from, to),$   
    PRECOND:  $At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$   
    EFFECT:  $\neg At(p, from) \wedge At(p, to)$ )

# PDDL – Example The Blocks World

---

$Init(On(A, Table) \wedge On(B, Table) \wedge On(C, A)$   
     $\wedge Block(A) \wedge Block(B) \wedge Block(C) \wedge Clear(B) \wedge Clear(C))$   
 $Goal(On(A, B) \wedge On(B, C))$   
 $Action(Move(b, x, y),$   
    PRECOND:  $On(b, x) \wedge Clear(b) \wedge Clear(y) \wedge Block(b) \wedge Block(y) \wedge$   
         $(b \neq x) \wedge (b \neq y) \wedge (x \neq y),$   
    EFFECT:  $On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y))$   
 $Action(MoveToTable(b, x),$   
    PRECOND:  $On(b, x) \wedge Clear(b) \wedge Block(b) \wedge (b \neq x),$   
    EFFECT:  $On(b, Table) \wedge Clear(x) \wedge \neg On(b, x))$



# Planning vs. General Search

---

## □ Fundamental difference is in representation

- States/situations – descriptions of the world by logical formulae
  - Agents can explicitly reason about and communicate with the world
- Actions/Operators – Transformation on formulae in the logical form
  - Agents can gauge the effect of actions by inspecting the EFFECT
- Goal Conditions – logical formulae vs. goal test
  - Agent can reflect on its goals

# Planning as Search

---

- ❑ Forward Search
- ❑ Backward Search

# Forward (Progression) Search

---

## ❑ Initial state – set of positive ground literals

- Literals not appearing are false

## ❑ Actions

- Application if preconditions are satisfied
- Add positive effect literals
- Remove negative effect literals

## ❑ Goal test: Does state logically satisfy goal?

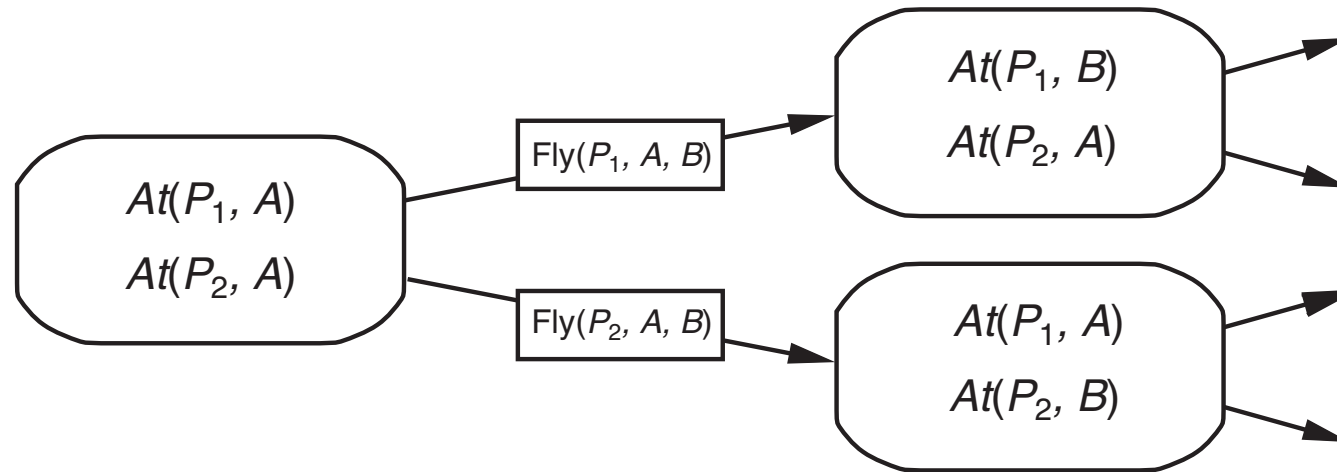
## ❑ Step cost: 1 (per action step)

## ❑ Uninformed (DFS/BFS), Informed ( $A^*$ ) and Local Search

## ❑ Domain independent heuristics can be derived

# Forward (Progression) Search (2)

---



- The branching factor can be prohibitively high, making the search inefficient.
- Prone to exploring irrelevant actions
- Sound procedure
  - Will result in consistent states

# Forward (Progression) Search (3)

---

## ❑ Consider the air cargo problem

- 10 airports, each airport has 5 planes and 20 pieces of cargo
- Goal is to move all cargo at airport A to airport B

## ❑ Simplest solution will be

- Load the 20 pieces of cargo into one of the planes at A
- Fly plane A to B
- Unload the cargo

## ❑ However, finding this solution can be difficult as the average branching factor is huge

- Each of the 50 planes can fly to 9 other airports
- Each of the 200 pieces of package can be either loaded or unloaded
- And so on...
- On average there are about 2000 possible actions per state and the search graph up to the depth of the obvious solution has about  $2000^{41}$  nodes!

# Backward (Regression) Search

---

## □ Relevant state search

- Consider actions relevant to the goal(current state)
- A set of relevant states to be considered in each step

## □ Start at goal state

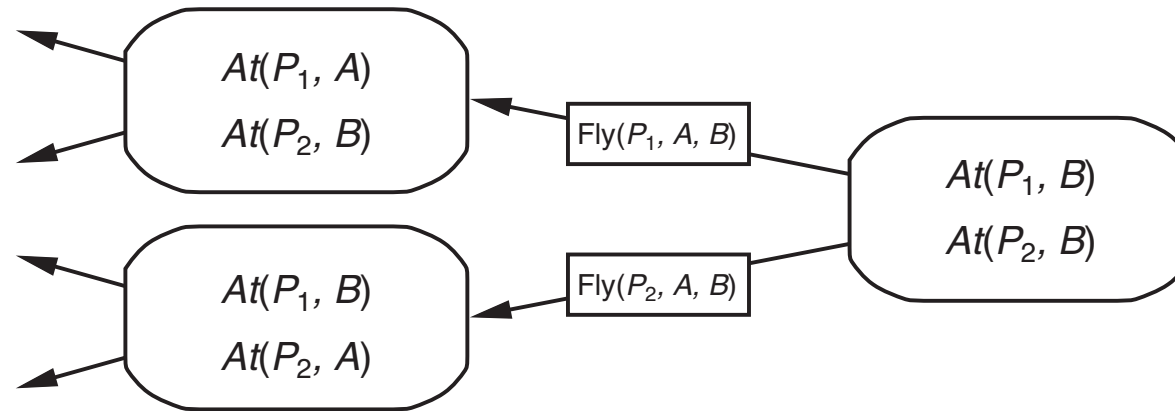
- Apply actions backward until we find a sequence of steps that reaches the initial state
- Goal – conjunction of literals
- Works only when we know how to regress from a state description to the predecessor state description

## □ In PDDL

- $g' = (g - ADD(a)) \cup Precond(a)$ 
  - $g$ - goal description
  - $a$ - ground action

# Backward (Regression) Search (2)

---



# Backward (Regression) Search (3)

---

❑ We need to also deal with partially uninstantiated actions and states (not just ground ones)

- For example: goal is  $At(C2, SFO)$
- Suggests action  $Unload(C2, p', SFO)$

$Action(Unload(C2, p', SFO),$   
 $PRECOND: In(C2, p') \wedge At(p', SFO) \wedge Plane(p') \wedge Cargo(C2) \wedge Airport(SFO)$   
 $EFFECT: At(C2, SFO) \wedge \neg In(C2, p'))$

- The regressed state

$In(C2, p') \wedge At(p', SFO) \wedge Plane(p') \wedge Cargo(C2) \wedge Airport(SFO)$



# Backward (Regression) Search (4)

---

- ❑ Actions that could be the last step in a plan leading up to the current goal
  - Relevant actions – actions that must contribute to the goal
  - Action's effect (either positive or negative) must unify with an element of the goal.
- ❑ However, action must not have any effect that negates an element of the goal
  - Goal –  $A \wedge B \wedge C$  and an action has the effect  $A \wedge B \wedge \neg C$

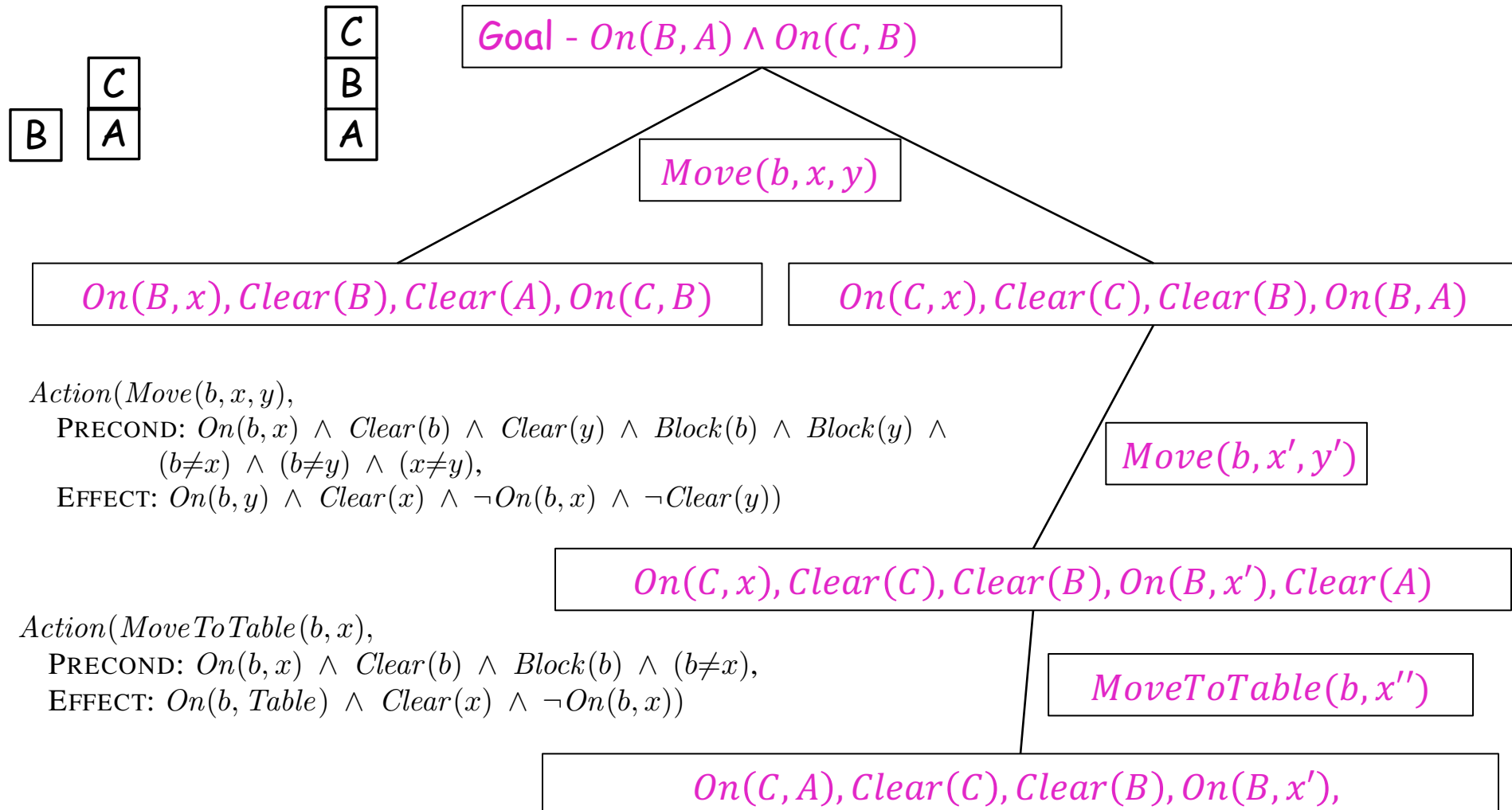
# Backward (Regression) Search (5)

---

## □ For Example

- Goal is *Own*(1234567890)
- An initial state with 1 billion ISBNs
- Single action scheme  
$$A = \text{Action}(\text{Buy}(i), \text{PRECOND: ISBN}(i), \text{EFFECT: OWN}(i))$$
- Unify goal *Own*(1234567890) with standardized effect *OWN*( $i'$ ), producing  $\theta = \{i'/1234567890\}$
- Regress over the action to produce the predecessor state description *ISBN*(1234567890), that is part of the initial state.

# Backward (Regression) Search (6)



# Planning Search

---

## ❑ Forward (Progression) Search

- Applicable actions
- Branching factor is high
- Search is sound
  - Will result in consistent states

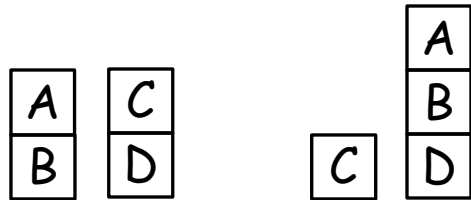
## ❑ Backward (Regression) Search

- Relevant actions
- Branching factor is low
- Search is not sound (not closed)
  - Can result in spurious states
    - Some check to remove such states

Can we combine the advantage of Progression and Regression search? – sound and a small branching factor?

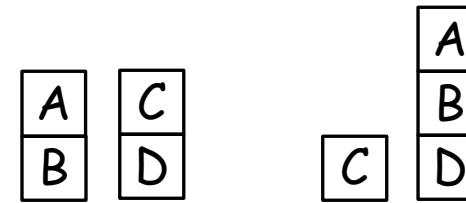
# Goal Stack Planning

- ❑ Goal directed search, but plan is constructed in forward manner.
- ❑ Uses a stack during the search



- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action  $a$  onto the stack
        - ❖ Push  $\text{Precond}(a)$  onto the stack
        - ❖ Push each predicate of  $\text{Precond}(a)$  on to the stack
    - Else, element is an action
      - Add action  $a$  to the plan  $\Pi = \Pi + a$

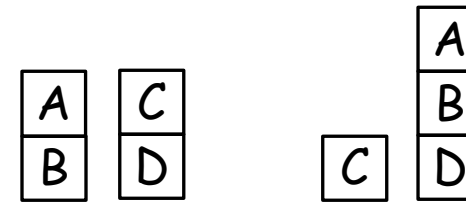
# Goal Stack Planning



- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action  $a$  onto the stack
        - ❖ Push  $\text{Precond}(a)$  onto the stack
        - ❖ Push each predicate of  $\text{Precond}(a)$  on to the stack
    - Else, element is an action
      - Add action  $a$  to the plan  $\Pi = \Pi + a$

|              |
|--------------|
| On AB        |
| On BD        |
| On AB, On BD |

# Goal Stack Planning

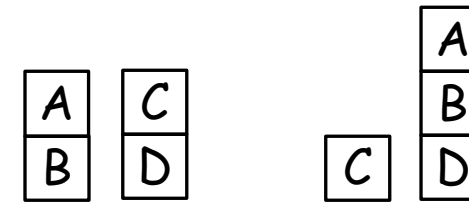


- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action  $a$  onto the stack
        - ❖ Push Precond( $a$ ) onto the stack
        - ❖ Push each predicate of Precond( $a$ ) on to the stack
    - Else, element is an action
      - Add action  $a$  to the plan  $\Pi = \Pi + a$

$Action(Move(b, x, y),$   
 PRECOND:  $On(b, x) \wedge Clear(b) \wedge Clear(y) \wedge Block(b) \wedge Block(y) \wedge$   
 $(b \neq x) \wedge (b \neq y) \wedge (x \neq y),$   
 EFFECT:  $On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y))$

|                  |
|------------------|
| <del>On AB</del> |
| On BD            |
| On AB, On BD     |

# Goal Stack Planning



*Action(MoveToTable(b, x),*  
 PRECOND:  $On(b, x) \wedge Clear(b) \wedge Block(b) \wedge (b \neq x)$ ,  
 EFFECT:  $On(b, Table) \wedge Clear(x) \wedge \neg On(b, x)$ )

❑ Push goals onto stack

❑ Push each predicate onto the stack

❑ While stack is not empty

○ Pop element from stack

➤ If element is a conjunct goal

▪ Select an ordering for subgoals and push them onto the stack

➤ Else If element is a predicate

▪ If true, do nothing

▪ Else

❖ Push relevant action  $a$  onto the stack

❖ Push Precond( $a$ ) onto the stack

❖ Push each predicate of Precond( $a$ ) on to the stack

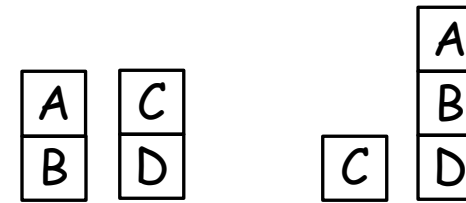
➤ Else, element is an action

▪ Add action  $a$  to the plan  $\Pi = \Pi + a$

|                  |
|------------------|
| Clear D          |
| Clear B          |
| Clear B, Clear D |
| Move BD          |
| <del>On AB</del> |
| <del>On BD</del> |
| On AB, On BD     |



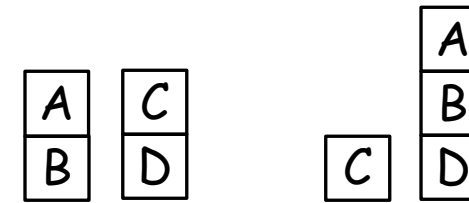
# Goal Stack Planning



|                    |
|--------------------|
| Clear C            |
| On CD              |
| On CD, Clear C     |
| Move T C           |
| <del>Clear D</del> |
| Clear B            |
| Clear B, Clear D   |
| Move BD            |
| <del>On AB</del>   |
| <del>On BD</del>   |
| On AB, On BD       |

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning



|                    |
|--------------------|
| <del>Clear C</del> |
| On CD              |
| On CD, Clear C     |
| Move T C           |
| <del>Clear D</del> |
| Clear B            |
| Clear B, Clear D   |
| Move BD            |
| <del>On AB</del>   |
| <del>On BD</del>   |
| On AB, On BD       |

❑ Push goals onto stack

❑ Push each predicate onto the stack

❑ While stack is not empty

○ Pop element from stack

➤ If element is a conjunct goal

▪ Select an ordering for subgoals and push them onto the stack

➤ Else If element is a predicate

▪ If true, do nothing

▪ Else

❖ Push relevant action a onto the stack

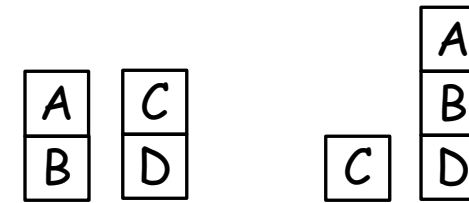
❖ Push Precond(a) onto the stack

❖ Push each predicate of Precond(a) on to the stack

➤ Else, element is an action

▪ Add action a to the plan  $\Pi = \Pi + a$

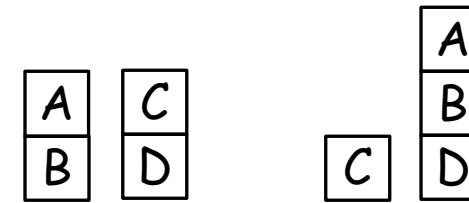
# Goal Stack Planning



|                    |
|--------------------|
| <del>Clear C</del> |
| <del>On CD</del>   |
| On CD, Clear C     |
| Move T C           |
| <del>Clear D</del> |
| Clear B            |
| Clear B, Clear D   |
| Move BD            |
| <del>On AB</del>   |
| <del>On BD</del>   |
| On AB, On BD       |

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning



|                           |
|---------------------------|
| <del>Clear C</del>        |
| <del>On CD</del>          |
| <del>On CD, Clear C</del> |
| Move T C                  |
| <del>Clear D</del>        |
| Clear B                   |
| Clear B, Clear D          |
| Move BD                   |
| <del>On AB</del>          |
| <del>On BD</del>          |
| On AB, On BD              |

❑ Push goals onto stack

❑ Push each predicate onto the stack

❑ While stack is not empty

○ Pop element from stack

➤ If element is a conjunct goal

▪ Select an ordering for subgoals and push them onto the stack

➤ Else If element is a predicate

▪ If true, do nothing

▪ Else

❖ Push relevant action a onto the stack

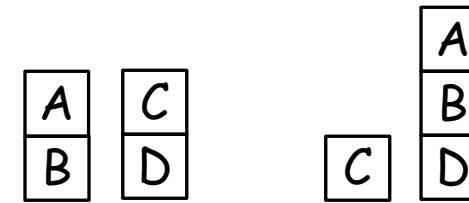
❖ Push Precond(a) onto the stack

❖ Push each predicate of Precond(a) on to the stack

➤ Else, element is an action

▪ Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning



|                           |
|---------------------------|
| <del>Clear C</del>        |
| <del>On CD</del>          |
| <del>On CD, Clear C</del> |
| <del>Move TC</del>        |
| <del>Clear D</del>        |
| Clear B                   |
| Clear B, Clear D          |
| Move BD                   |
| <del>On AB</del>          |
| <del>On BD</del>          |
| On AB, On BD              |

❑ Push goals onto stack

❑ Push each predicate onto the stack

❑ While stack is not empty

○ Pop element from stack

➤ If element is a conjunct goal

▪ Select an ordering for subgoals and push them onto the stack

➤ Else If element is a predicate

▪ If true, do nothing

▪ Else

❖ Push relevant action a onto the stack

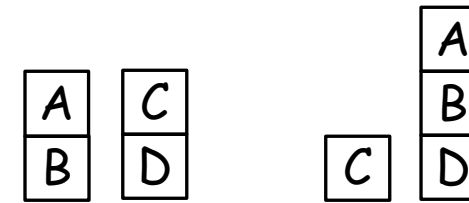
❖ Push Precond(a) onto the stack

❖ Push each predicate of Precond(a) on to the stack

➤ Else, element is an action

▪ Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning

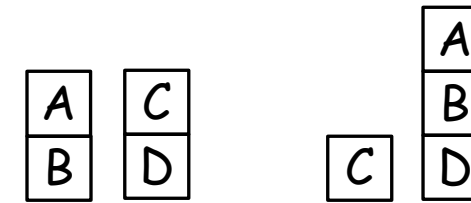


|                           |
|---------------------------|
| <del>Clear C</del>        |
| <del>On CD</del>          |
| <del>On CD, Clear C</del> |
| <del>Move TC</del>        |
| <del>Clear D</del>        |
| Clear B                   |
| Clear B, Clear D          |
| Move BD                   |
| <del>On AB</del>          |
| <del>On BD</del>          |
| On AB, On BD              |

Move TC

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning



$Action(MoveToTable(b, x),$   
 $PRECOND: On(b, x) \wedge Clear(b) \wedge Block(b) \wedge (b \neq x),$   
 $EFFECT: On(b, Table) \wedge Clear(x) \wedge \neg On(b, x))$

❑ Push goals onto stack

❑ Push each predicate onto the stack

❑ While stack is not empty

○ Pop element from stack

➤ If element is a conjunct goal

▪ Select an ordering for subgoals and push them onto the stack

➤ Else If element is a predicate

▪ If true, do nothing

▪ Else

❖ Push relevant action a onto the stack

❖ Push Precond(a) onto the stack

❖ Push each predicate of Precond(a) on to the stack

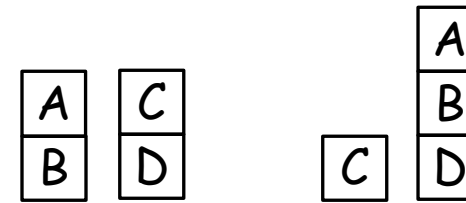
➤ Else, element is an action

▪ Add action a to the plan  $\Pi = \Pi + a$

|                    |
|--------------------|
| <del>Clear D</del> |
| Clear B            |
| Clear B, Clear D   |
| Move BD            |
| <del>On AB</del>   |
| <del>On BD</del>   |
| On AB, On BD       |

MoveT C

# Goal Stack Planning



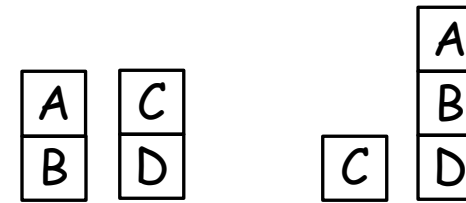
|                    |
|--------------------|
| Clear A            |
| On AB              |
| On AB, Clear A     |
| MoveT A            |
| <del>Clear D</del> |
| <del>Clear B</del> |
| Clear B, Clear D   |
| Move BD            |
| <del>On AB</del>   |
| <del>On BD</del>   |
| On AB, On BD       |

MoveT C

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$



# Goal Stack Planning

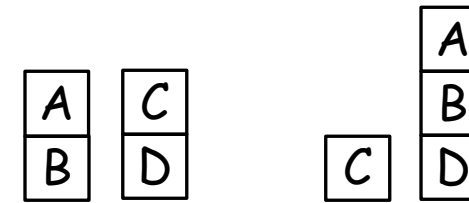


|                    |
|--------------------|
| <del>Clear A</del> |
| On AB              |
| On AB, Clear A     |
| MoveT A            |
| <del>Clear D</del> |
| <del>Clear B</del> |
| Clear B, Clear D   |
| Move BD            |
| <del>On AB</del>   |
| <del>On BD</del>   |
| On AB, On BD       |

MoveT C

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning

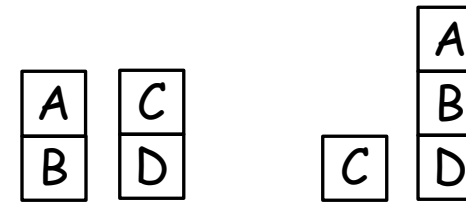


|                    |
|--------------------|
| <del>Clear A</del> |
| <del>On AB</del>   |
| On AB, Clear A     |
| Move T A           |
| <del>Clear D</del> |
| <del>Clear B</del> |
| Clear B, Clear D   |
| Move BD            |
| <del>On AB</del>   |
| <del>On BD</del>   |
| On AB, On BD       |

Move T C

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning

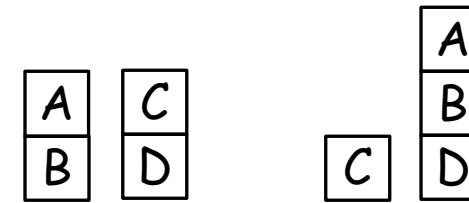


|                           |
|---------------------------|
| <del>Clear A</del>        |
| <del>On AB</del>          |
| <del>On AB, Clear A</del> |
| Move T A                  |
| <del>Clear D</del>        |
| <del>Clear B</del>        |
| Clear B, Clear D          |
| Move BD                   |
| <del>On AB</del>          |
| <del>On BD</del>          |
| On AB, On BD              |

Move T C

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning

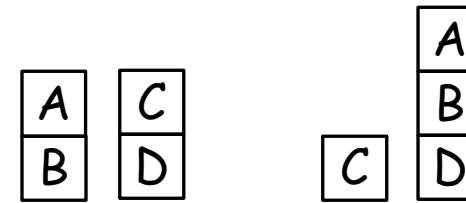


|                           |
|---------------------------|
| <del>Clear A</del>        |
| <del>On AB</del>          |
| <del>On AB, Clear A</del> |
| <del>Move T A</del>       |
| <del>Clear D</del>        |
| <del>Clear B</del>        |
| Clear B, Clear D          |
| Move BD                   |
| <del>On AB</del>          |
| <del>On BD</del>          |
| On AB, On BD              |

Move T C

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning

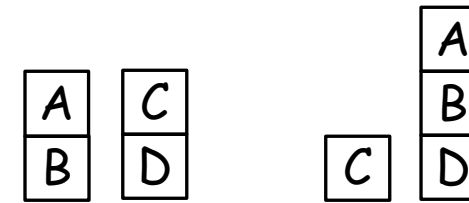


|                           |
|---------------------------|
| <del>Clear A</del>        |
| <del>On AB</del>          |
| <del>On AB, Clear A</del> |
| <del>Move T A</del>       |
| <del>Clear D</del>        |
| <del>Clear B</del>        |
| Clear B, Clear D          |
| Move BD                   |
| <del>On AB</del>          |
| <del>On BD</del>          |
| On AB, On BD              |

|          |
|----------|
| Move T A |
| Move T C |

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning

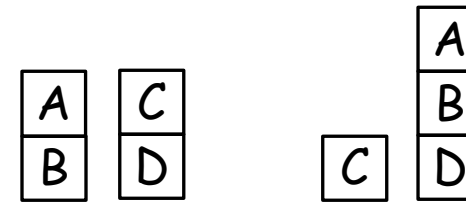


- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

|                  |
|------------------|
| Clear B, Clear D |
| Move BD          |
| <del>On AB</del> |
| <del>On BD</del> |
| On AB, On BD     |

|         |
|---------|
| MoveT A |
| MoveT C |

# Goal Stack Planning

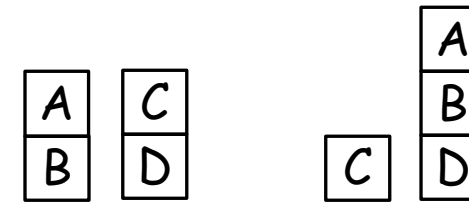


- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

|                             |
|-----------------------------|
| <del>Clear B, Clear D</del> |
| Move BD                     |
| <del>On AB</del>            |
| <del>On BD</del>            |
| On AB, On BD                |

|         |
|---------|
| MoveT A |
| MoveT C |

# Goal Stack Planning



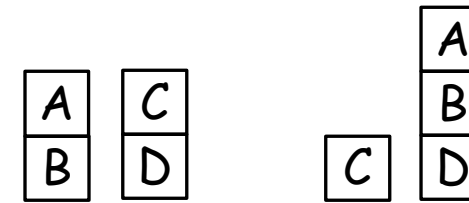
- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

|                             |
|-----------------------------|
| <del>Clear B, Clear D</del> |
| <del>Move BD</del>          |
| <del>On AB</del>            |
| <del>On BD</del>            |
| On AB, On BD                |

|         |
|---------|
| MoveT A |
| MoveT C |



# Goal Stack Planning

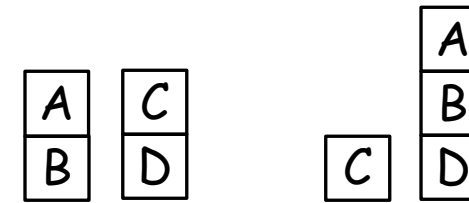


- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

|                             |
|-----------------------------|
| <del>Clear B, Clear D</del> |
| <del>Move BD</del>          |
| <del>On AB</del>            |
| <del>On BD</del>            |
| On AB, On BD                |

|         |
|---------|
| Move BD |
| MoveT A |
| MoveT C |

# Goal Stack Planning



$Action(Move(b, x, y),$   
 $PRECOND: On(b, x) \wedge Clear(b) \wedge Clear(y) \wedge Block(b) \wedge Block(y) \wedge$   
 $(b \neq x) \wedge (b \neq y) \wedge (x \neq y),$   
 $EFFECT: On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y))$

|                             |
|-----------------------------|
| On AB                       |
| On BD                       |
| <del>Clear B, Clear D</del> |
| <del>Move BD</del>          |
| <del>On AB</del>            |
| <del>On BD</del>            |
| On AB, On BD                |

|          |
|----------|
| Move BD  |
| Move T A |
| Move T C |

❑ Push goals onto stack

❑ Push each predicate onto the stack

❑ While stack is not empty

○ Pop element from stack

➤ If element is a conjunct goal

▪ Select an ordering for subgoals and push them onto the stack

➤ Else If element is a predicate

▪ If true, do nothing

▪ Else

❖ Push relevant action a onto the stack

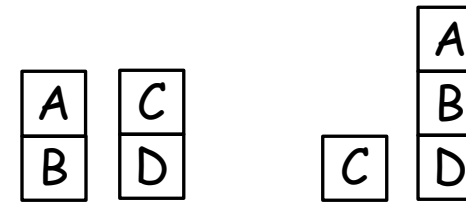
❖ Push Precond(a) onto the stack

❖ Push each predicate of Precond(a) on to the stack

➤ Else, element is an action

▪ Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning

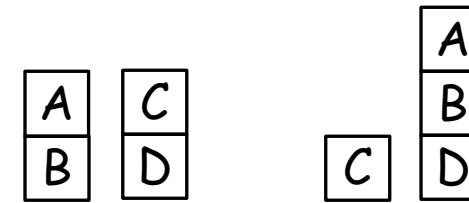


|                             |
|-----------------------------|
| Clear B                     |
| Clear A                     |
| Clear A, Clear B            |
| Move AB                     |
| <del>On AB</del>            |
| On BD                       |
| <del>Clear B, Clear D</del> |
| <del>Move BD</del>          |
| <del>On AB</del>            |
| <del>On BD</del>            |
| On AB, On BD                |

|         |
|---------|
| Move BD |
| MoveT A |
| MoveT C |

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning

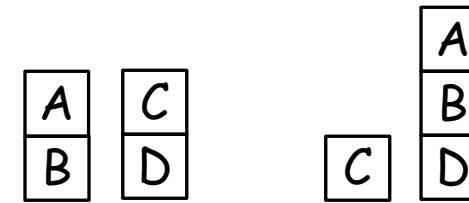


|                             |
|-----------------------------|
| <del>Clear B</del>          |
| Clear A                     |
| Clear A, Clear B            |
| Move AB                     |
| <del>On AB</del>            |
| On BD                       |
| <del>Clear B, Clear D</del> |
| <del>Move BD</del>          |
| <del>On AB</del>            |
| <del>On BD</del>            |
| On AB, On BD                |

|         |
|---------|
| Move BD |
| MoveT A |
| MoveT C |

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning

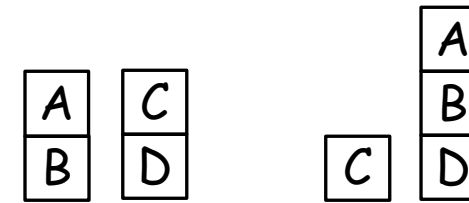


|                             |
|-----------------------------|
| <del>Clear B</del>          |
| <del>Clear A</del>          |
| Clear A, Clear B            |
| Move AB                     |
| <del>On AB</del>            |
| On BD                       |
| <del>Clear B, Clear D</del> |
| <del>Move BD</del>          |
| <del>On AB</del>            |
| <del>On BD</del>            |
| On AB, On BD                |

|         |
|---------|
| Move BD |
| MoveT A |
| MoveT C |

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning

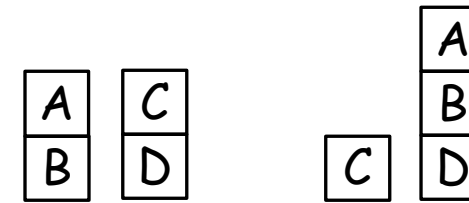


|                             |
|-----------------------------|
| <del>Clear B</del>          |
| <del>Clear A</del>          |
| <del>Clear A, Clear B</del> |
| Move AB                     |
| <del>On AB</del>            |
| On BD                       |
| <del>Clear B, Clear D</del> |
| <del>Move BD</del>          |
| <del>On AB</del>            |
| <del>On BD</del>            |
| On AB, On BD                |

|         |
|---------|
| Move BD |
| MoveT A |
| MoveT C |

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning



|                             |
|-----------------------------|
| <del>Clear B</del>          |
| <del>Clear A</del>          |
| <del>Clear A, Clear B</del> |
| <del>Move AB</del>          |
| <del>On AB</del>            |
| On BD                       |
| <del>Clear B, Clear D</del> |
| <del>Move BD</del>          |
| <del>On AB</del>            |
| <del>On BD</del>            |
| On AB, On BD                |

|         |
|---------|
| Move BD |
| MoveT A |
| MoveT C |

❑ Push goals onto stack

❑ Push each predicate onto the stack

❑ While stack is not empty

○ Pop element from stack

➤ If element is a conjunct goal

▪ Select an ordering for subgoals and push them onto the stack

➤ Else If element is a predicate

▪ If true, do nothing

▪ Else

❖ Push relevant action a onto the stack

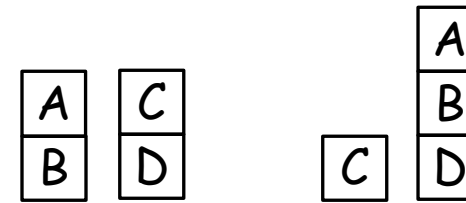
❖ Push Precond(a) onto the stack

❖ Push each predicate of Precond(a) on to the stack

➤ Else, element is an action

▪ Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning



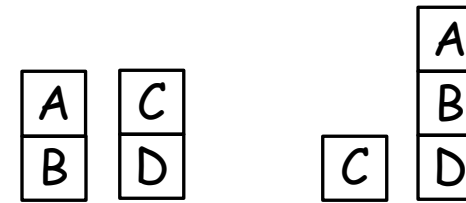
|                             |
|-----------------------------|
| <del>Clear B</del>          |
| <del>Clear A</del>          |
| <del>Clear A, Clear B</del> |
| <del>Move AB</del>          |
| <del>On AB</del>            |
| On BD                       |
| <del>Clear B, Clear D</del> |
| <del>Move BD</del>          |
| <del>On AB</del>            |
| <del>On BD</del>            |
| On AB, On BD                |

|         |
|---------|
| Move AB |
| Move BD |
| MoveT A |
| MoveT C |

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$



# Goal Stack Planning

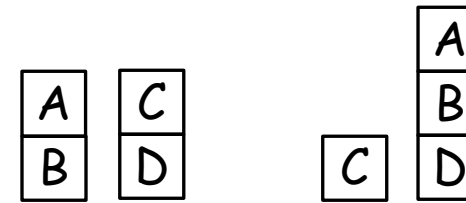


|                             |
|-----------------------------|
| <del>Clear B</del>          |
| <del>Clear A</del>          |
| <del>Clear A, Clear B</del> |
| <del>Move AB</del>          |
| <del>On AB</del>            |
| <del>On BD</del>            |
| <del>Clear B, Clear D</del> |
| <del>Move BD</del>          |
| <del>On AB</del>            |
| <del>On BD</del>            |
| On AB, On BD                |

|         |
|---------|
| Move AB |
| Move BD |
| MoveT A |
| MoveT C |

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning



|                             |
|-----------------------------|
| <del>Clear B</del>          |
| <del>Clear A</del>          |
| <del>Clear A, Clear B</del> |
| <del>Move AB</del>          |
| <del>On AB</del>            |
| <del>On BD</del>            |
| <del>Clear B, Clear D</del> |
| <del>Move BD</del>          |
| <del>On AB</del>            |
| <del>On BD</del>            |
| <del>On AB, On BD</del>     |

|         |
|---------|
| Move AB |
| Move BD |
| MoveT A |
| MoveT C |

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action a onto the stack
        - ❖ Push Precond(a) onto the stack
        - ❖ Push each predicate of Precond(a) on to the stack
    - Else, element is an action
      - Add action a to the plan  $\Pi = \Pi + a$

# Goal Stack Planning

---

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element from stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action  $a$  onto the stack
        - ❖ Push  $\text{Precond}(a)$  onto the stack
        - ❖ Push each predicate of  $\text{Precond}(a)$  on to the stack
    - Else, element is an action
      - Add action  $a$  to the plan  $\Pi = \Pi + a$

- ❑ Sound?
  - All solutions are legal plans
- ❑ Complete?
  - May not find a solution if one exists
  - Give an Example – Exercise
- ❑ Optimality?
  - Suboptimal – Sussman's Anomaly

# Goal Stack Planning – Sussman's Anomaly

- ❑ There is a serializable order, which does not involve doing extra work

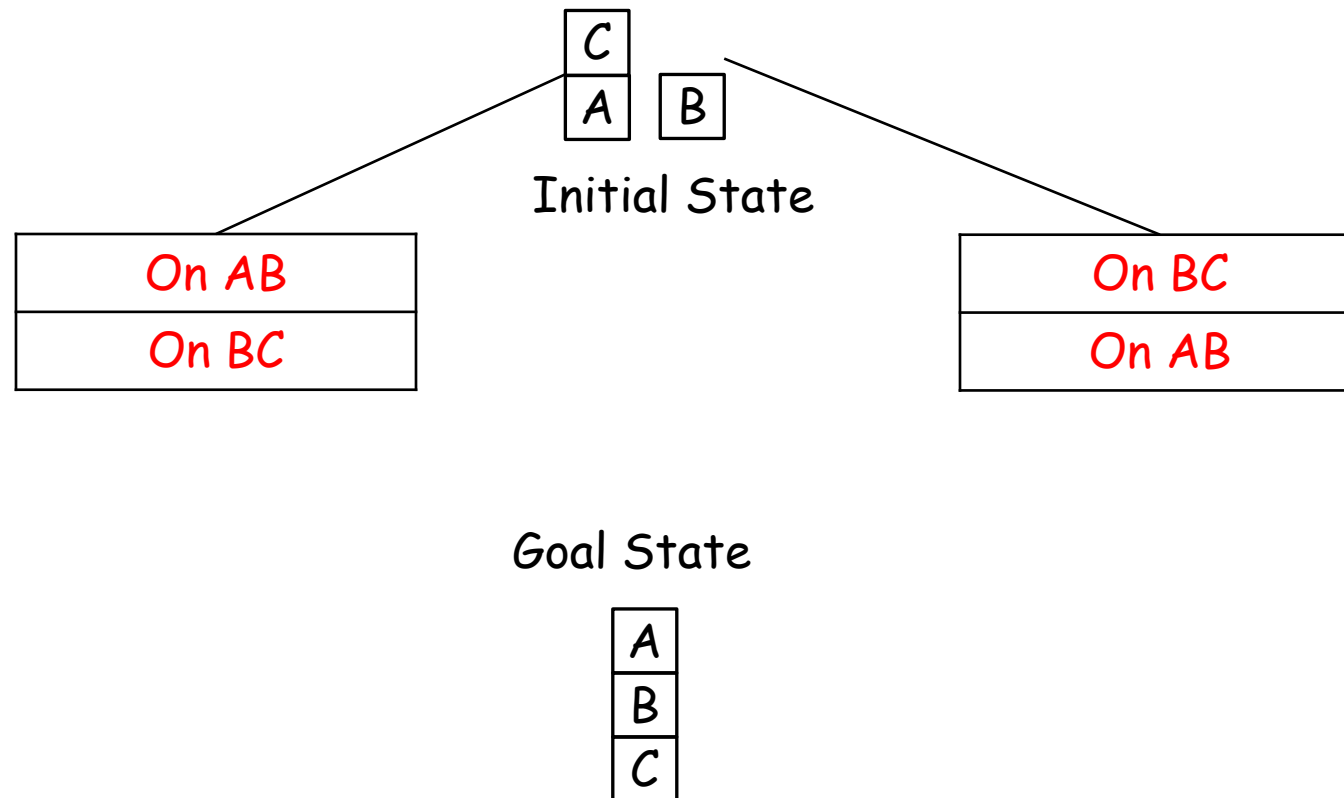
|                  |  |  |
|------------------|--|--|
| Clear B          |  |  |
| Clear A          |  |  |
| Clear A, Clear B |  |  |
| Move AB          |  |  |
| On AB            |  |  |
| On BD            |  |  |
| Clear B, Clear D |  |  |
| Move BD          |  |  |
| On AB            |  |  |
| On BD            |  |  |
| On AB, On BD     |  |  |

|         |
|---------|
| Move AB |
| Move BD |
| MoveT A |
| MoveT C |

|                  |
|------------------|
| Clear B          |
| Clear A          |
| Clear A, Clear B |
| Move AB          |
| Move BD          |
| On BD            |
| On AB            |
| On AB, On BD     |

# Goal Stack Planning – Sussman's Anomaly

❑ Is there a serializable order, which does not involve doing extra work?



# Goal Stack Planning

---

- ❑ Push goals onto stack
- ❑ Push each predicate onto the stack
- ❑ While stack is not empty
  - Pop element the stack
    - If element is a conjunct goal
      - Select an ordering for subgoals and push them onto the stack
    - Else If element is a predicate
      - If true, do nothing
      - Else
        - ❖ Push relevant action  $a$  onto the stack
        - ❖ Push  $\text{Precond}(a)$  onto the stack
        - ❖ Push each predicate of  $\text{Precond}(a)$  on to the stack
    - Else, element is an action
      - Add action  $a$  to the plan  $\Pi = \Pi + a$

- ❑ Sound?
  - All solutions are legal plans
- ❑ Complete?
  - May not find a solution if one exists
  - Give an Example – Exercise
- ❑ Optimality?
  - Suboptimal – Sussman's Anomaly
- ❑ Linear Planning
  - Serializable goals
- ❑ Non-Serializable goals
  - Non-Linear planning, aka least commitment planning aka planning space planning

# Heuristics for Planning

---

- ❑ Relaxing the problem, to solve an easier version of itself.
- ❑ Estimate solution length
  - Ignore some or all preconditions
    - Minimum number of actions required such that the union of whose effects satisfies the goal - Set cover problem NP-Hard
  - Ignore delete lists
    - Makes monotonic progress towards the goal – NP-Hard

# Graph Plan

---

## ❑ A propositional planner, no variables

- Simpler algorithm – no need for unification, however, is bulkier as all the propositions have to be enlisted

## ❑ Incremental planning algorithm

- Make a plan graph of depth  $k$
- Search for a solution
- If succeed return a plan
- Else,  $k=k+1$
- Repeat from step 1



# Planning Graph

---

## ❑ Directed Layered graph

- Alternate layers of propositions and actions

## ❑ Two types of nodes

- Proposition – P
- Action – A

## ❑ Three types of edges between levels

- Precondition
- Add
- Delete

## ❑ Action layer includes actions whose propositions (preconditions) may be satisfied in the previous level, and a no-op action

# Planning Graph (2)

*Init*(*Have*(*Cake*))

*Goal*(*Have*(*Cake*)  $\wedge$  *Eaten*(*Cake*))

*Action*(*Eat*(*Cake*))

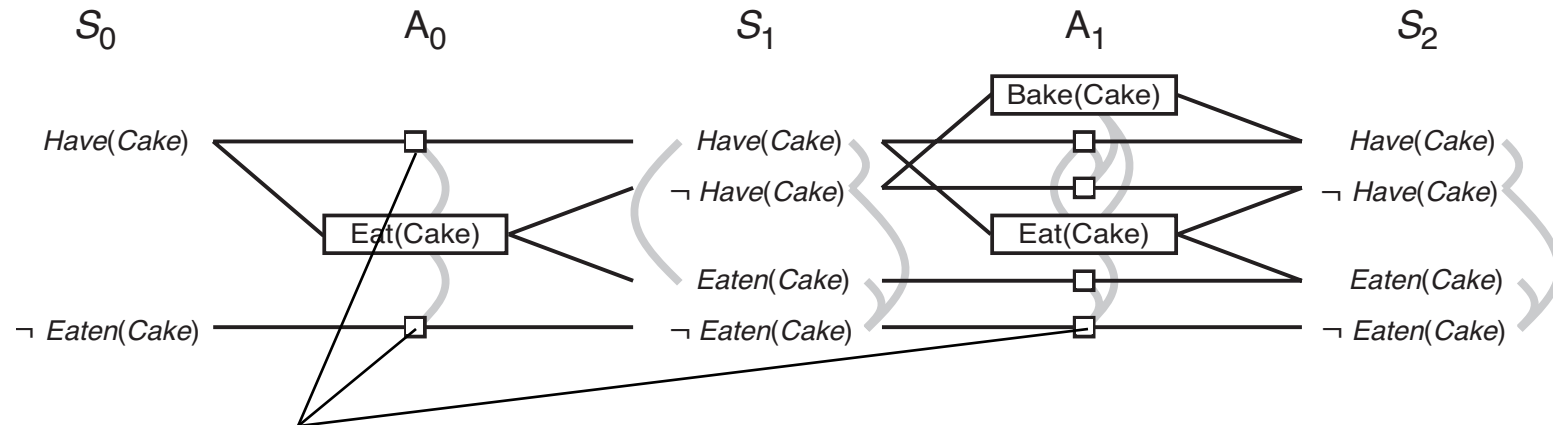
PRECOND: *Have*(*Cake*)

EFFECT:  $\neg$  *Have*(*Cake*)  $\wedge$  *Eaten*(*Cake*)

*Action*(*Bake*(*Cake*))

PRECOND:  $\neg$  *Have*(*Cake*)

EFFECT: *Have*(*Cake*)



Persistence action (no-op)

# Mutual Exclusion -Mutex Links

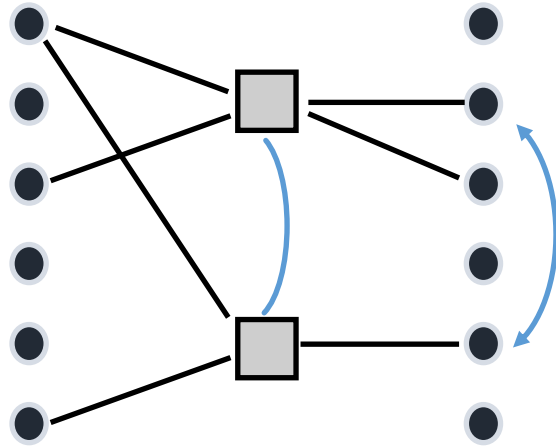
---

- ❑ Two actions are mutually exclusive (mutex) at some stage, if no valid plan could contain both
- ❑ Two actions are mutex if
  - Inconsistent effects: one action negates an effect of the other
  - Interference – one of the effects of one action is the negation of a precondition of the other.
  - Competing needs – one of the preconditions of one action is mutually exclusive with a precondition of the other
- ❑ Two propositions are mutex if
  - Inconsistent support- if one literal is the negation of the other or, if each possible pair of actions that could achieve the two literals is mutually exclusive.

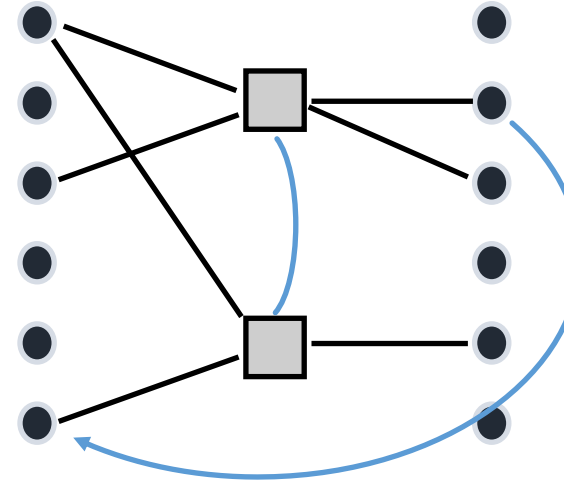
# Mutex Links

Jim Blythe and Dan Weld

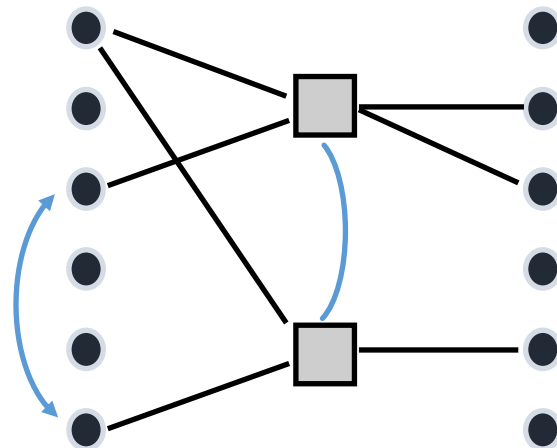
Inconsistent  
Effects



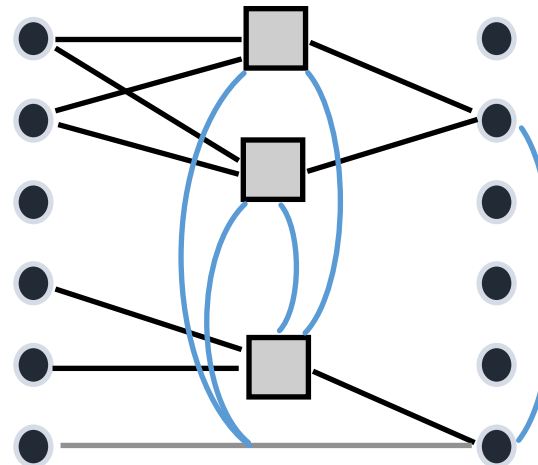
Interference



Competing  
Needs

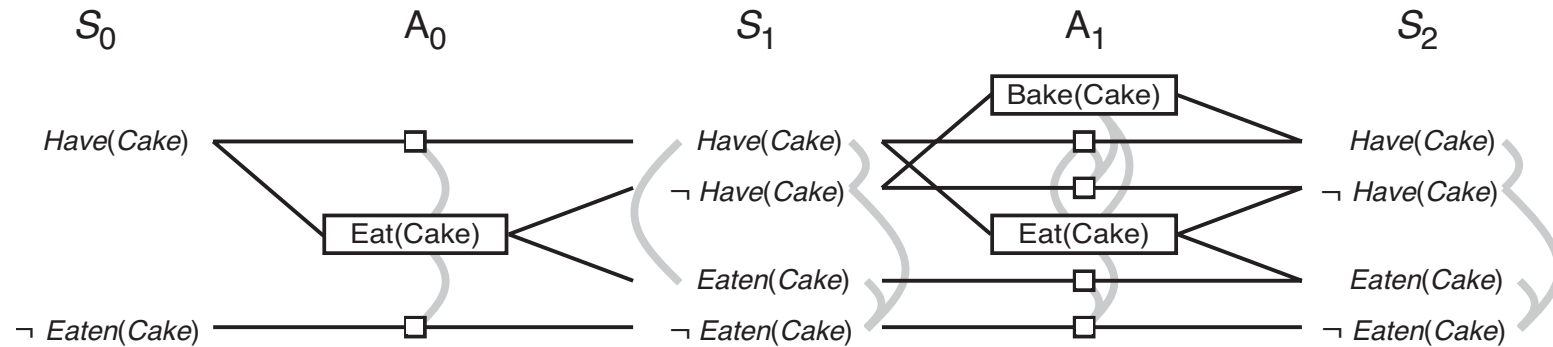


Inconsistent  
Support



# Mutex Links

$Init(Have(Cake))$   
 $Goal(Have(Cake) \wedge Eaten(Cake))$   
 $Action(Eat(Cake))$   
    PRECOND:  $Have(Cake)$   
    EFFECT:  $\neg Have(Cake) \wedge Eaten(Cake)$   
 $Action(Bake(Cake))$   
    PRECOND:  $\neg Have(Cake)$   
    EFFECT:  $Have(Cake)$



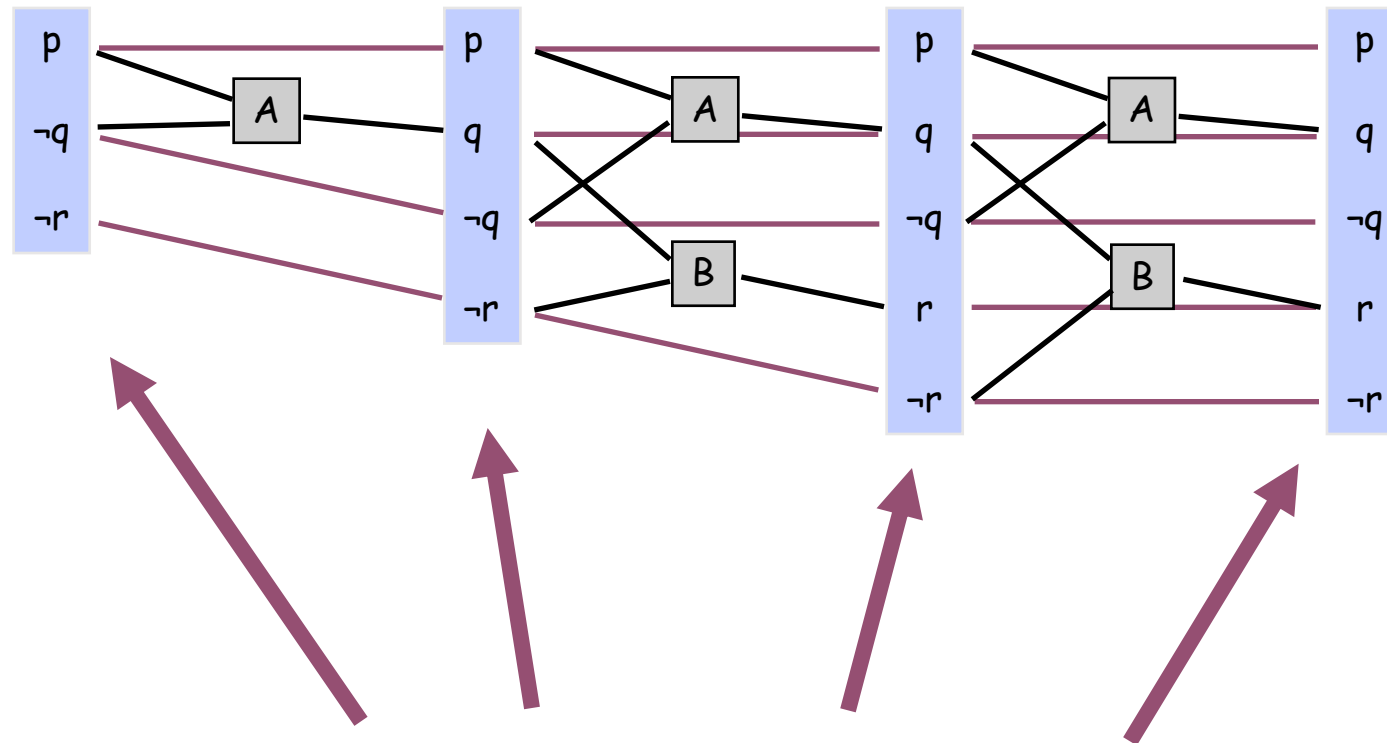
# Planning Graph (3)

---

- ❑  $L$  – literals
- ❑  $A$  – actions
- ❑  $S_i$  – no more than  $l$  nodes,  $l^2$  mutex links
- ❑  $A_i$  – no more than  $a + l$  nodes,  $(a + l)^2$  mutex links,  $2(al + l)$  precondition and effect links
- ❑ Total size of an entire graph with  $n$  levels –  $O(n(a + l)^2)$

# Observation 1

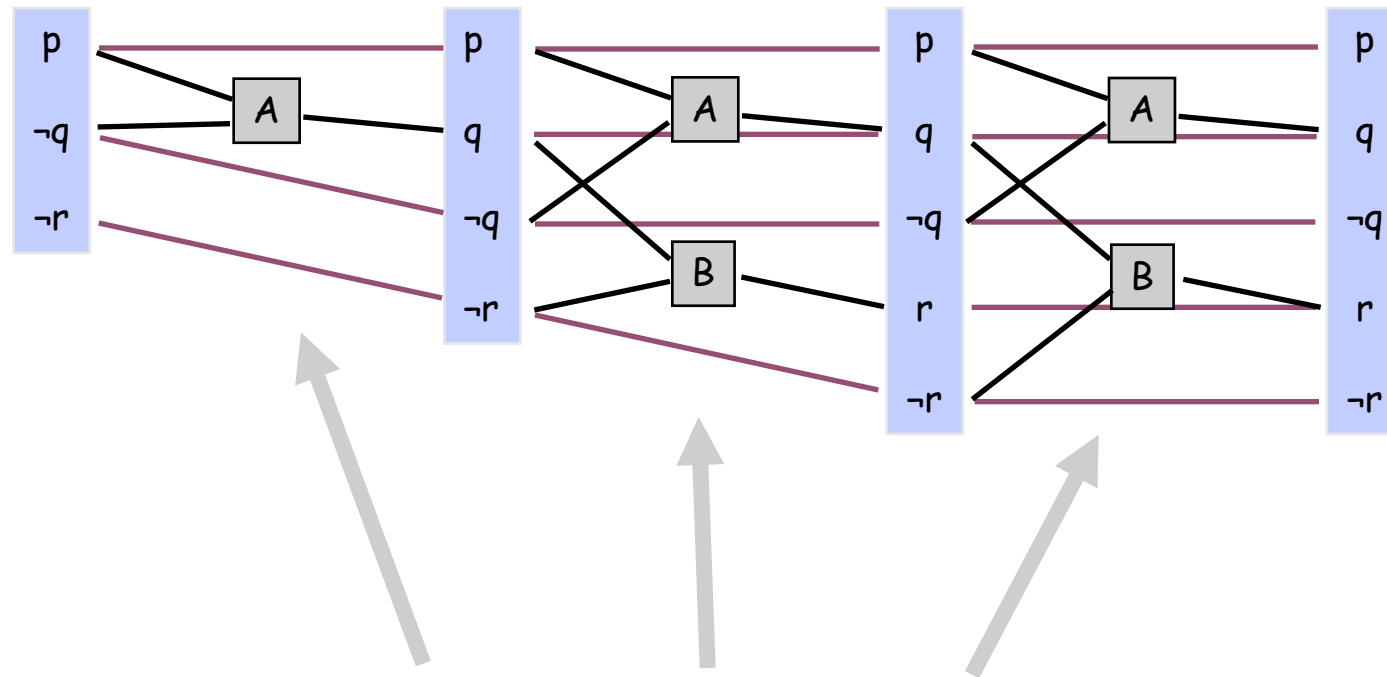
Jim Blythe and Dan Weld



Propositions monotonically increase  
(always carried forward by no-ops)

# Observation 2

Jim Blythe and Dan Weld

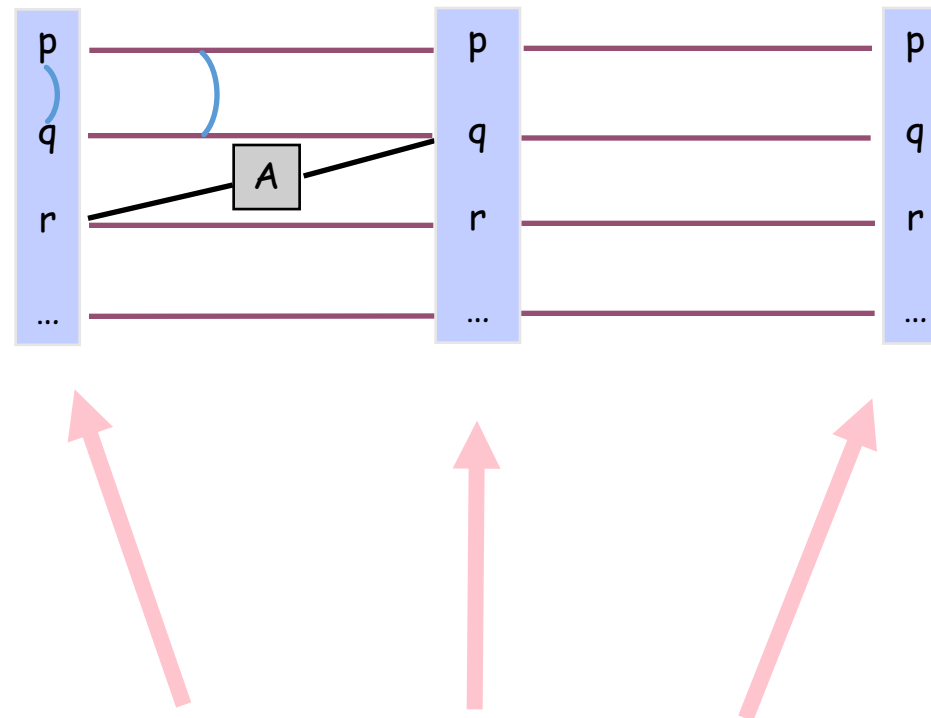


Actions monotonically increase



# Observation 3

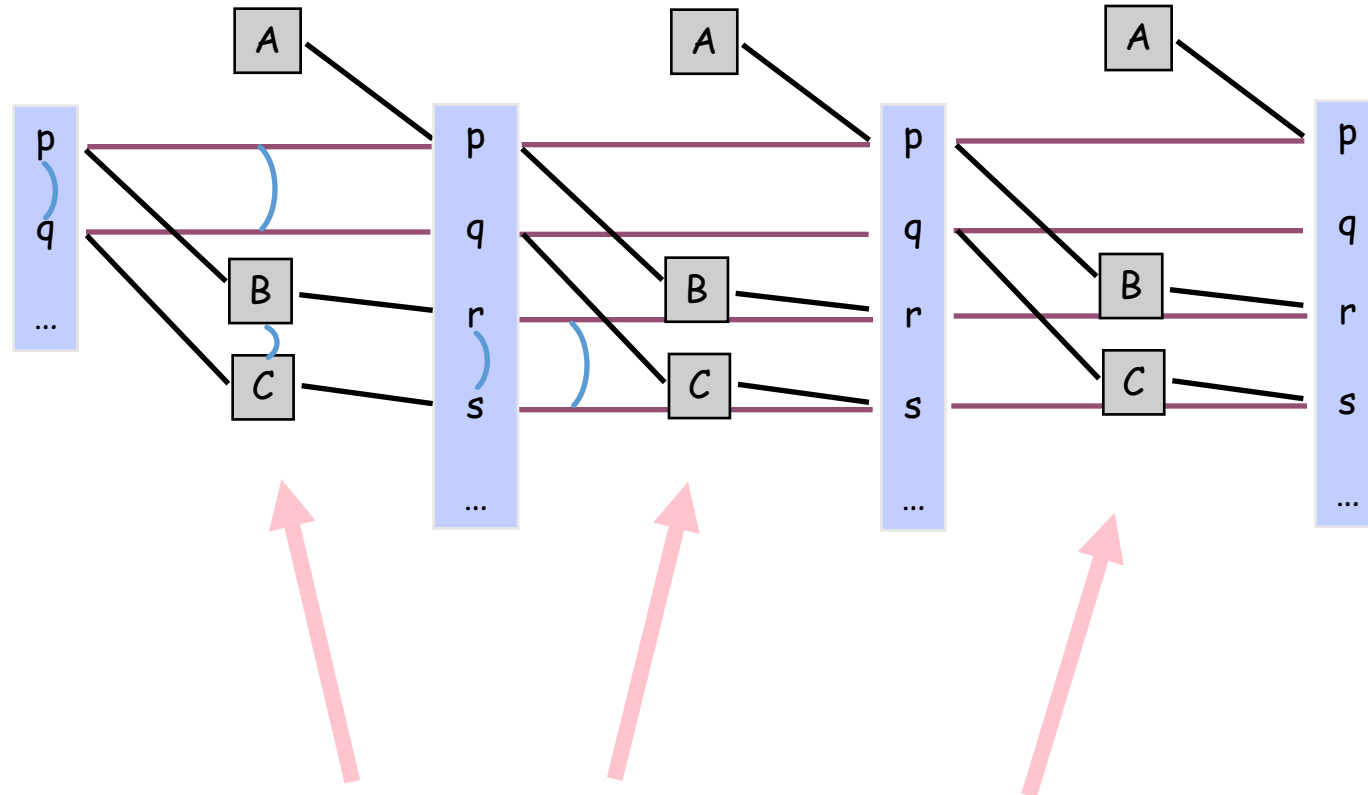
Jim Blythe and Dan Weld



Proposition mutex relationships monotonically decrease

# Observation 4

Jim Blythe and Dan Weld



Action mutex relationships monotonically decrease

# Observation 5

---

Jim Blythe and Dan Weld

## □ Planning Graph levels off

- After a certain  $k$ , all levels are identical

## □ Valid plan

- Actions at the same level do not interfere
- Each action's preconditions are made true by the plan
- Goals are satisfied (non mutex)

# Graph Plan Algorithm

---

```
function GRAPHPLAN(problem) returns solution or failure
  graph  $\leftarrow$  INITIAL-PLANNING-GRAPH(problem)
  goals  $\leftarrow$  CONJUNCTS(problem.GOAL)
  nogoods  $\leftarrow$  an empty hash table
  for tl = 0 to  $\infty$  do
    if goals all non-mutex in  $S_t$  of graph then
      solution  $\leftarrow$  EXTRACT-SOLUTION(graph, goals, NUMLEVELS(graph), nogoods)
      if solution  $\neq$  failure then return solution
    if graph and nogoods have both leveled off then return failure
    graph  $\leftarrow$  EXPAND-GRAPH(graph, problem)
```

# Searching for a plan

---

- ❑ Backward search on the planning graph
- ❑ Achieve goals level by level
- ❑ At level  $k$ , pick a subset of non-mutex actions to achieve current goals. Their preconditions become the goals for  $k-1$  level
- ❑ Build goal subset by picking each goal and choosing an action to add. Use one already selected if possible. Do forward checking on remaining goals (backtrack if can't pick non-mutex action)

# Birthday Dinner Example

---

❑ Initial Conditions: (and (garbage) (cleanHands) (quiet))

❑ Goal: (and (dinner) (present) (not (garbage)))

❑ Actions:

- Cook :precondition (cleanHands)  
:effect (dinner)
- Wrap :precondition (quiet)  
:effect (present)
- Carry :precondition  
:effect (and (not (garbage)) (not (cleanHands)))
- Dolly :precondition  
:effect (and (not (garbage)) (not (quiet)))

# Birthday Dinner Example

---

clean•

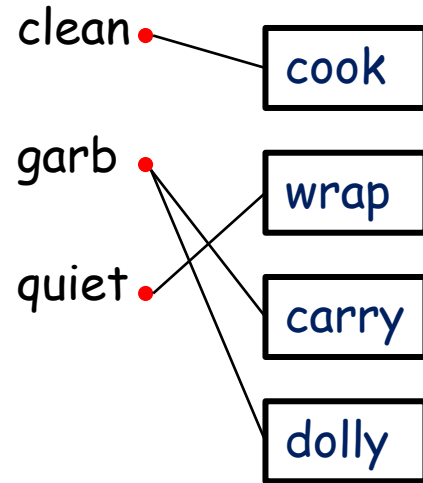
garb •

quiet•

|        |                                                               |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example

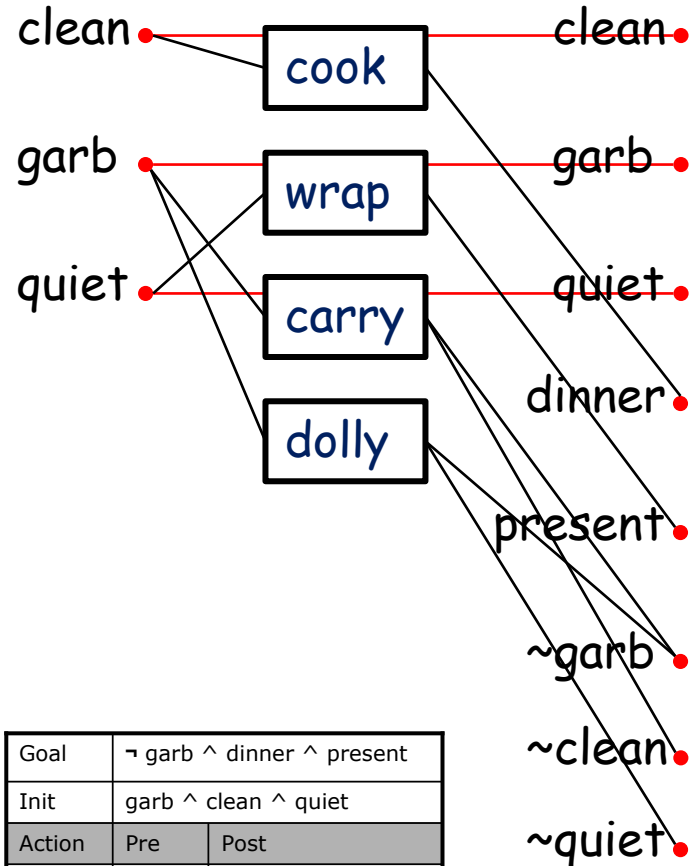
---



|        |                                                               |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

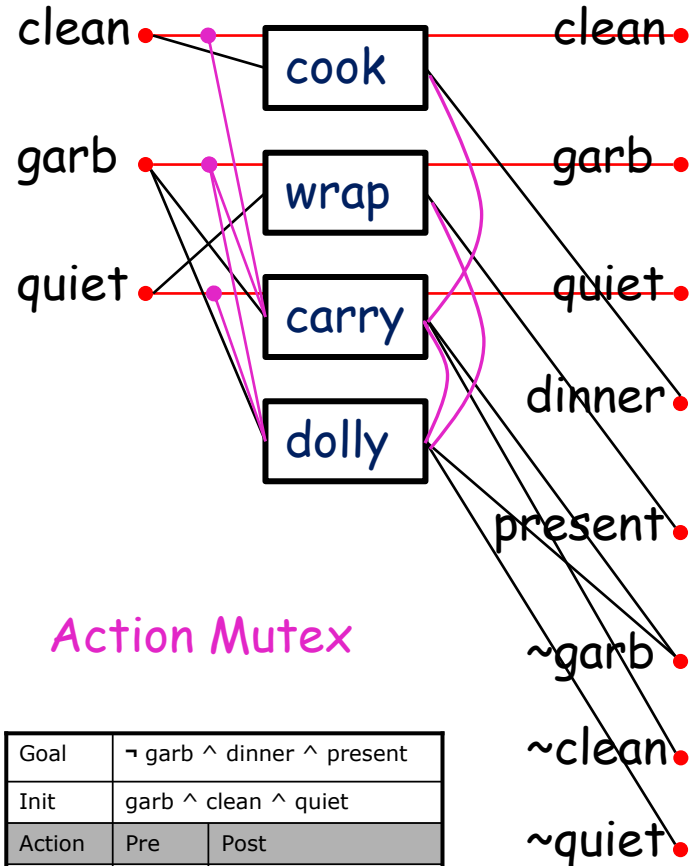


# Birthday Dinner Example



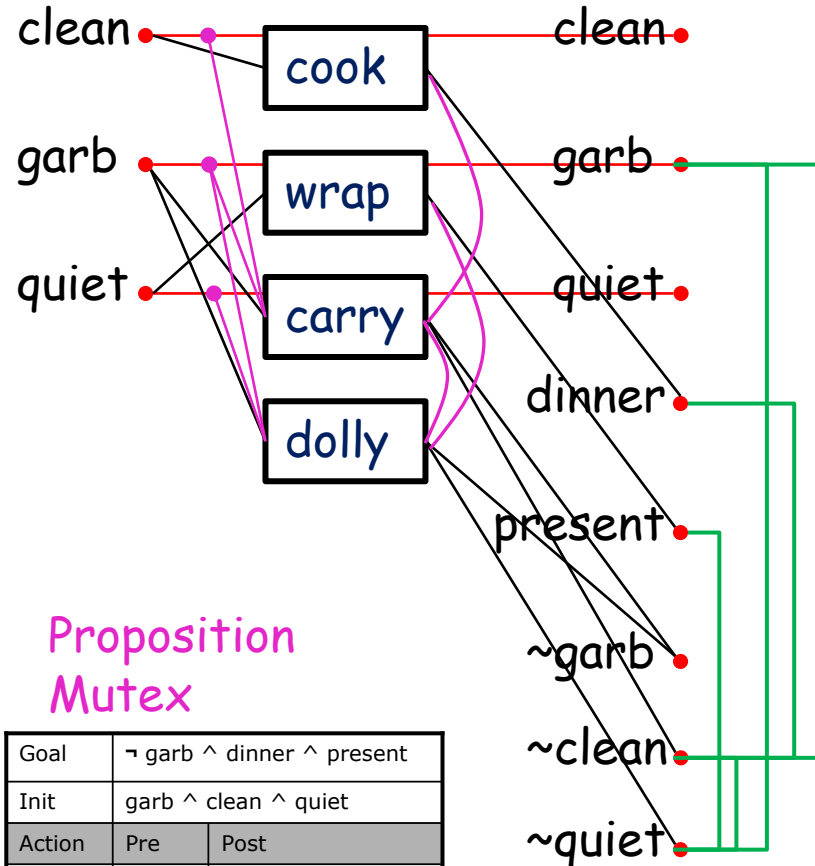
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example



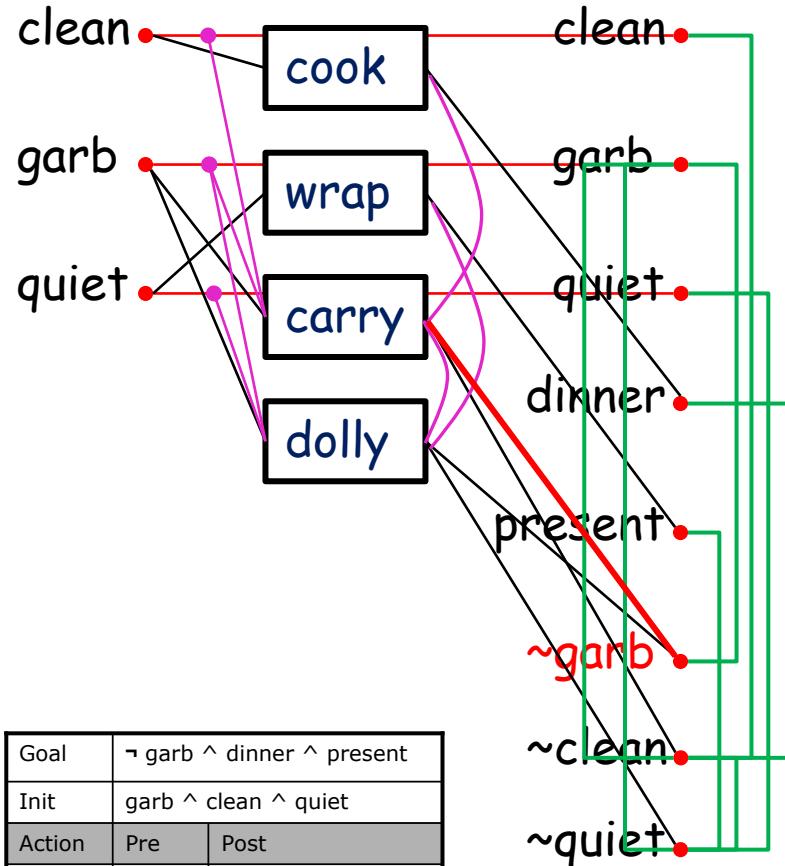
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example



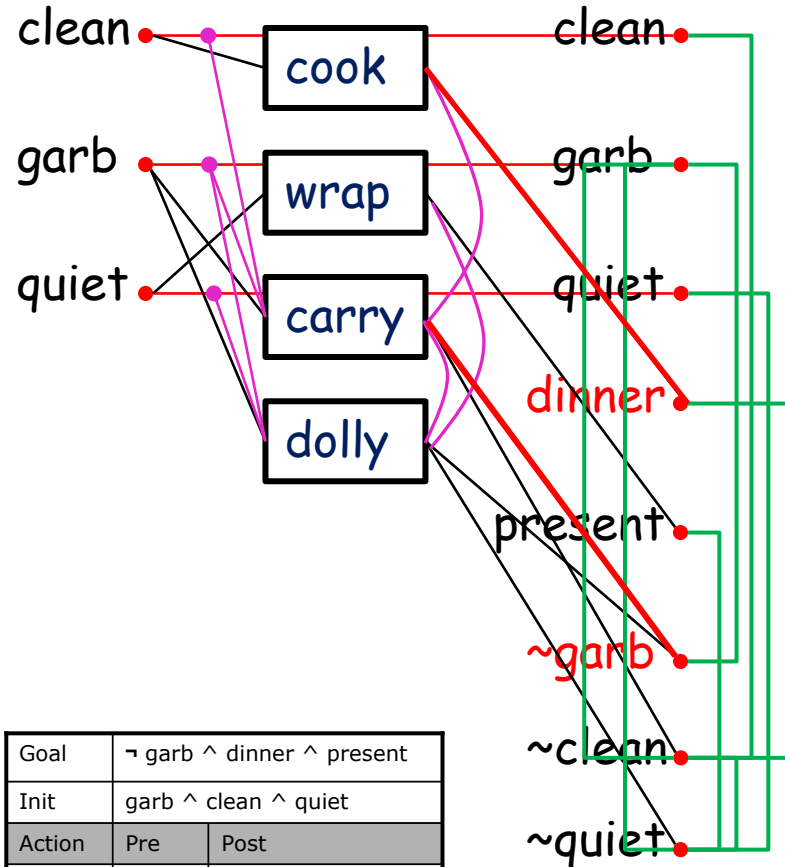
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example



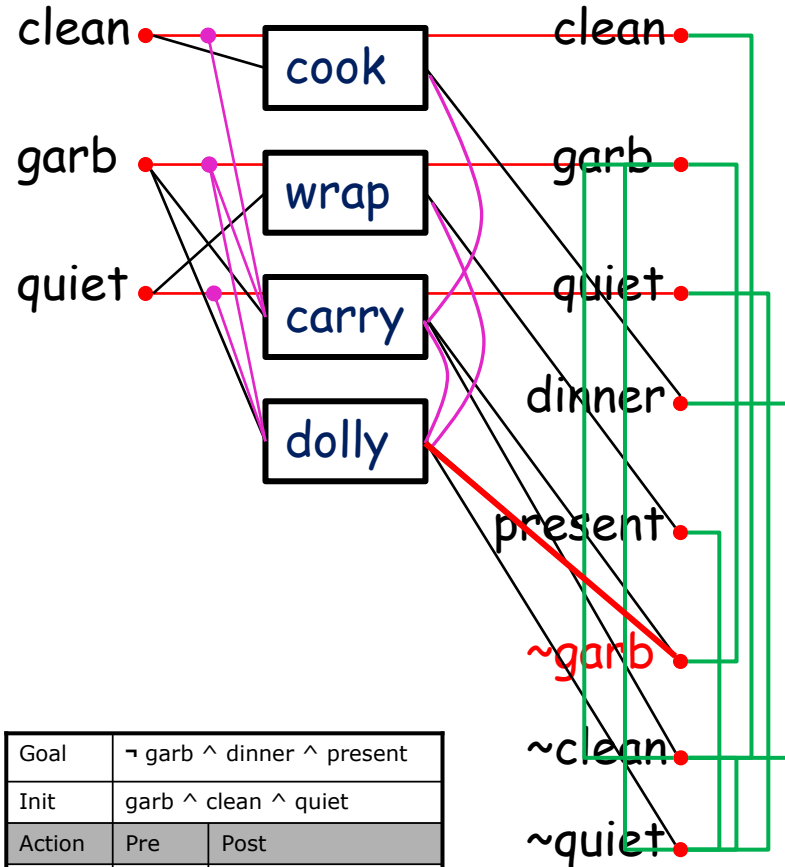
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example



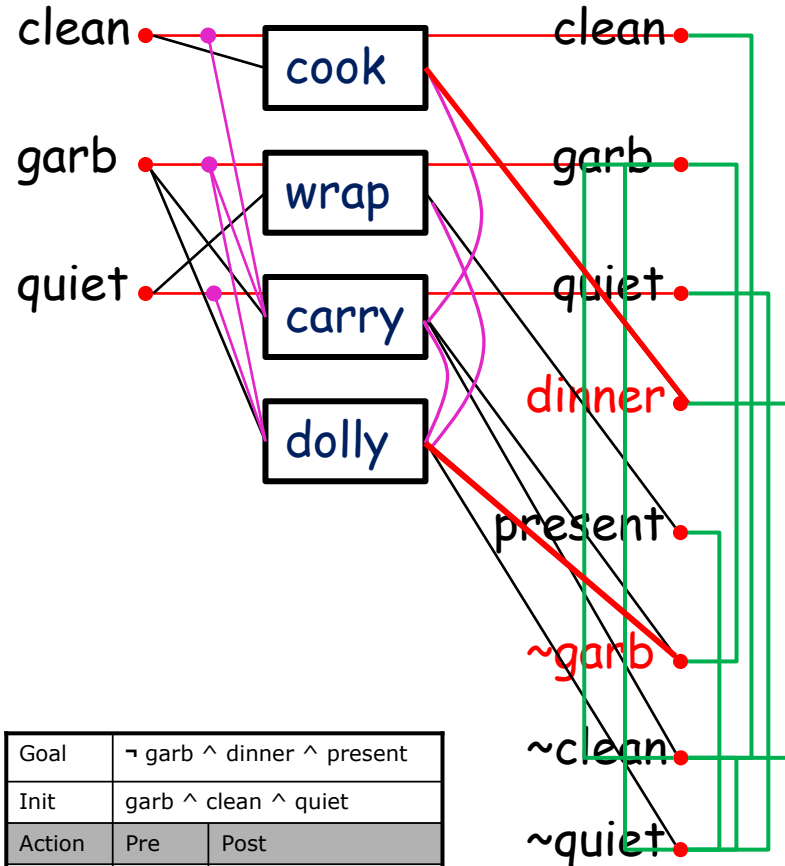
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example



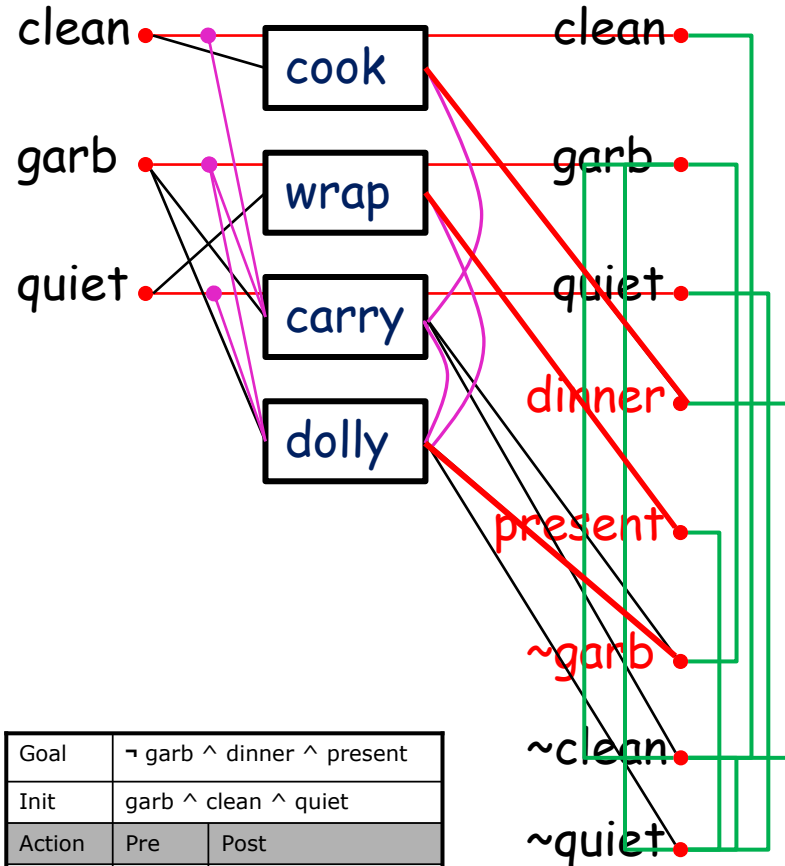
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example



| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

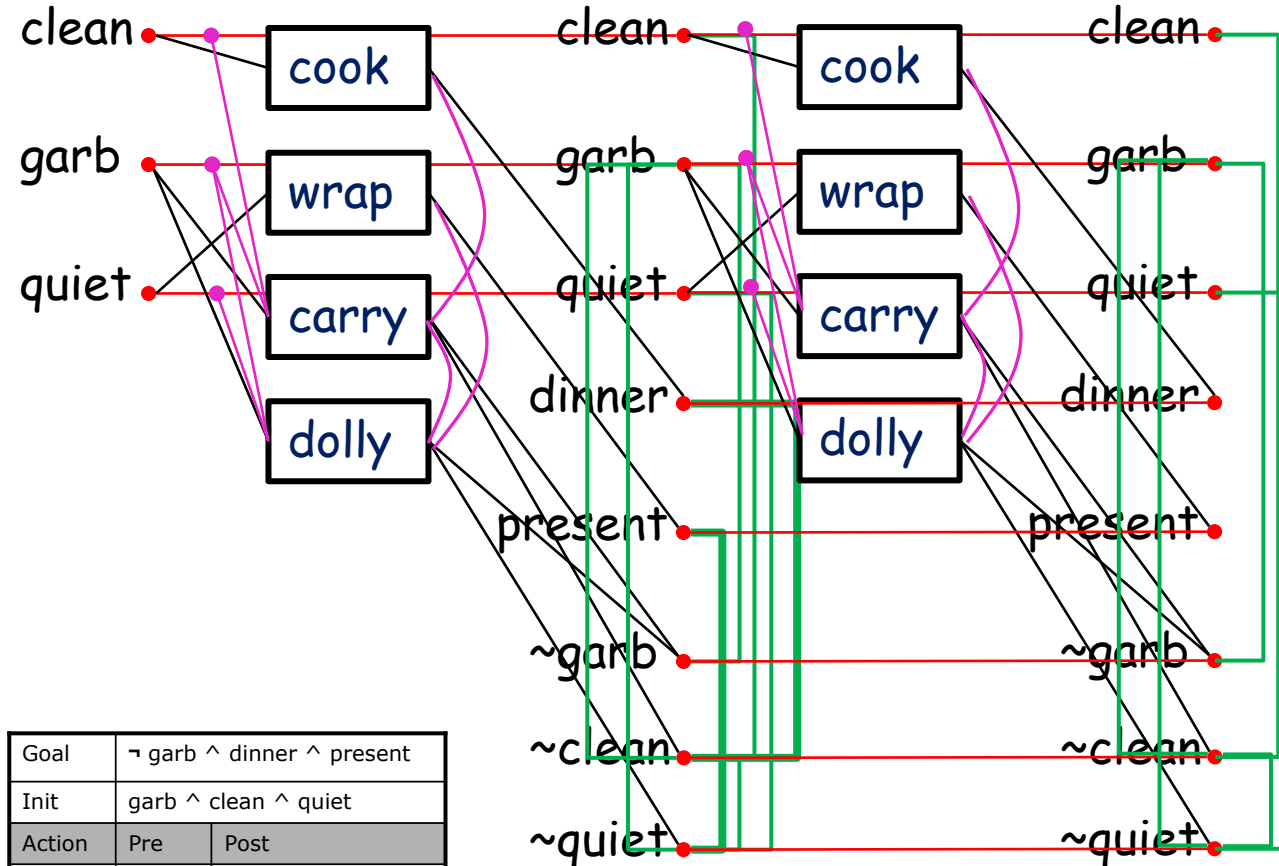
# Birthday Dinner Example



| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

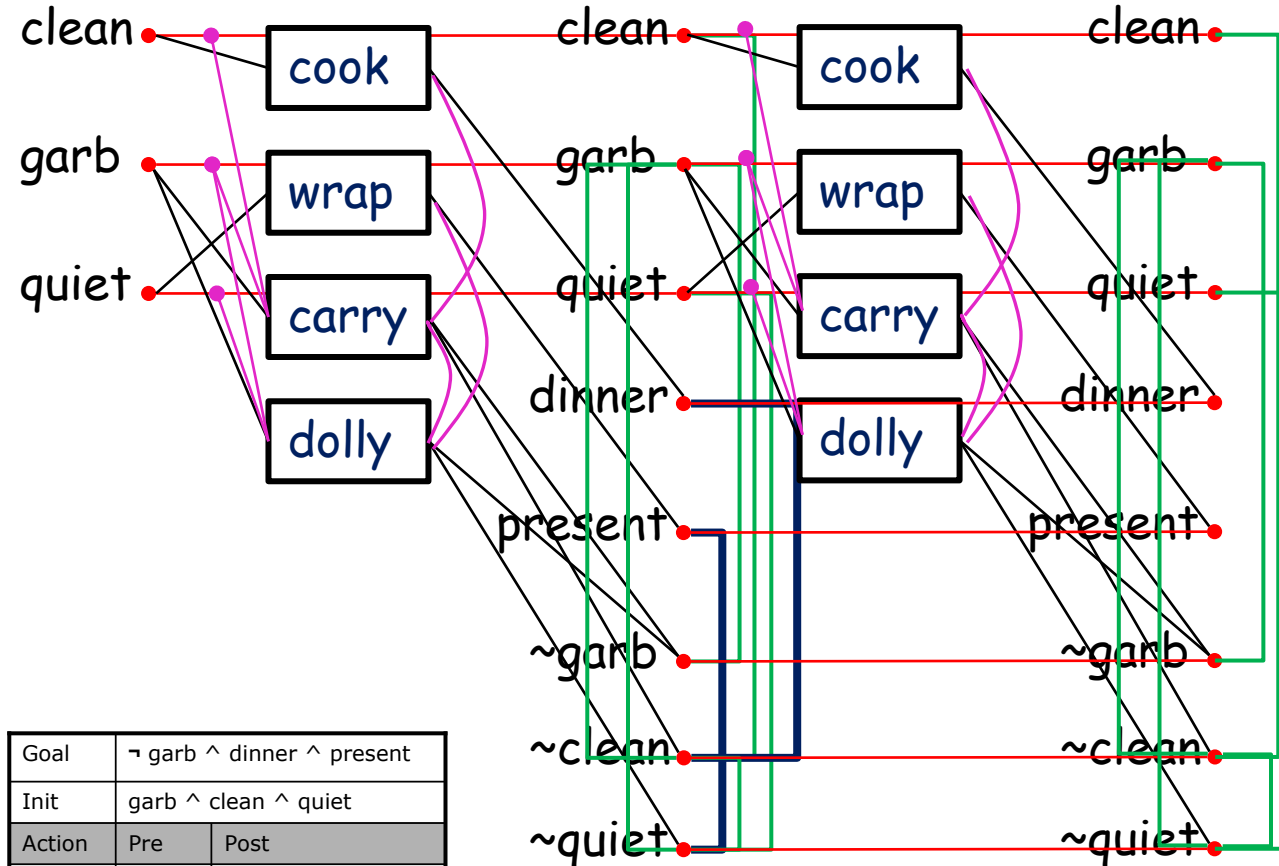


# Birthday Dinner Example



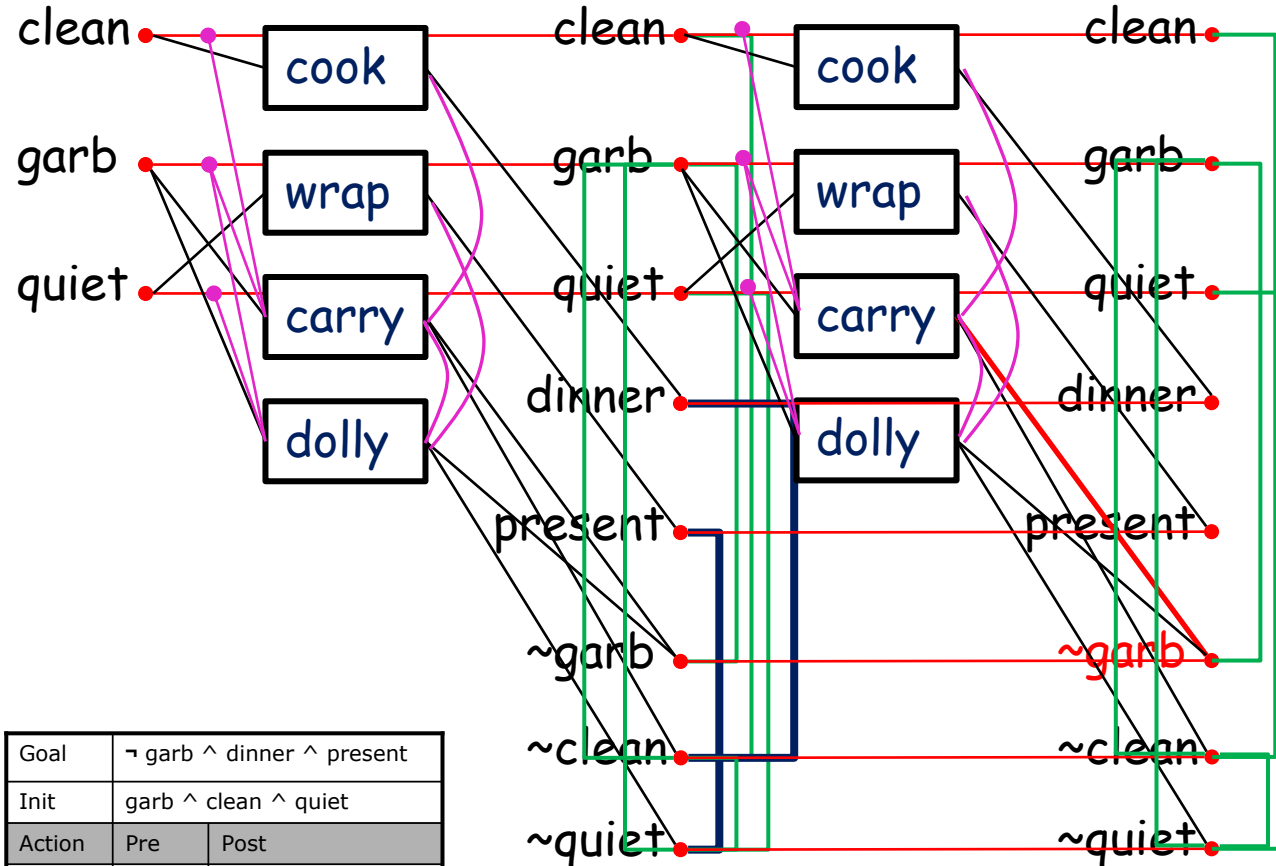
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example



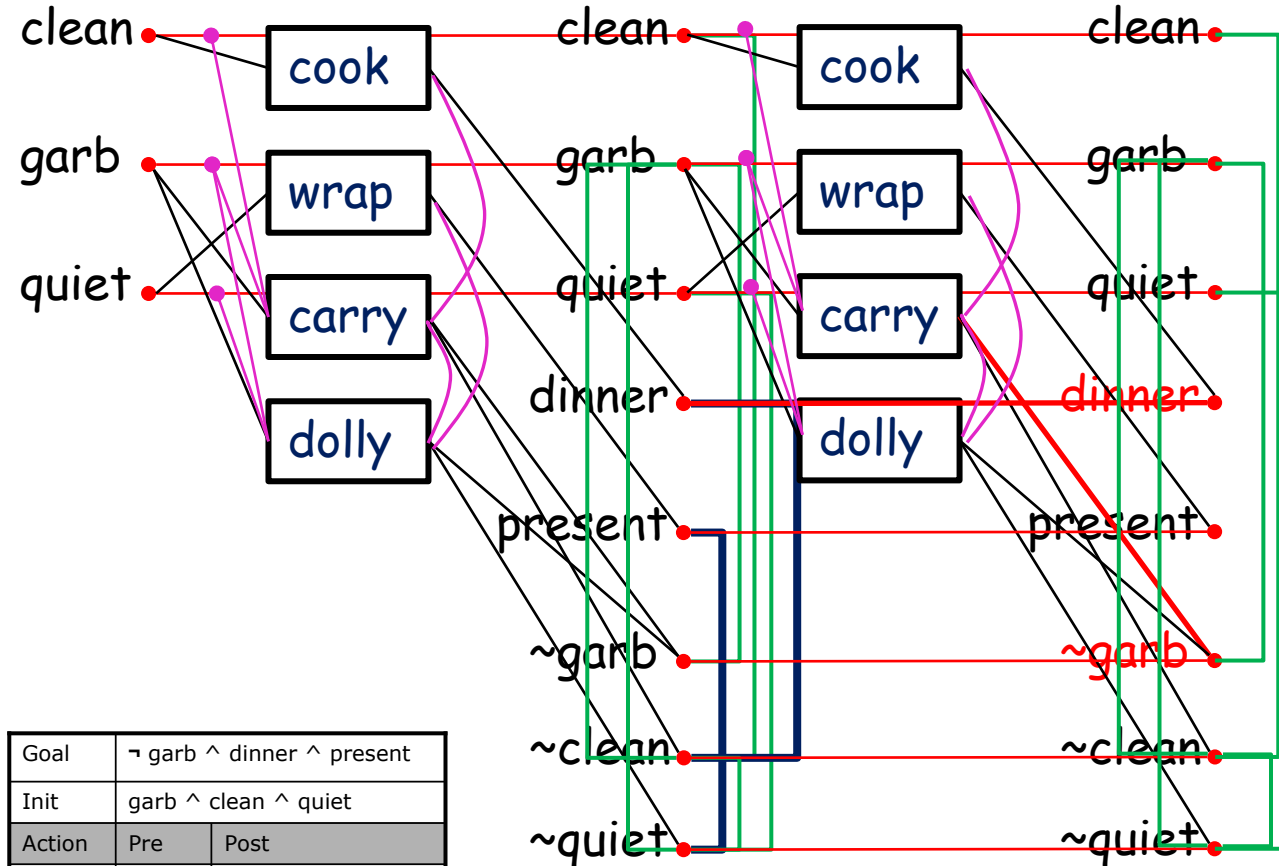
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example



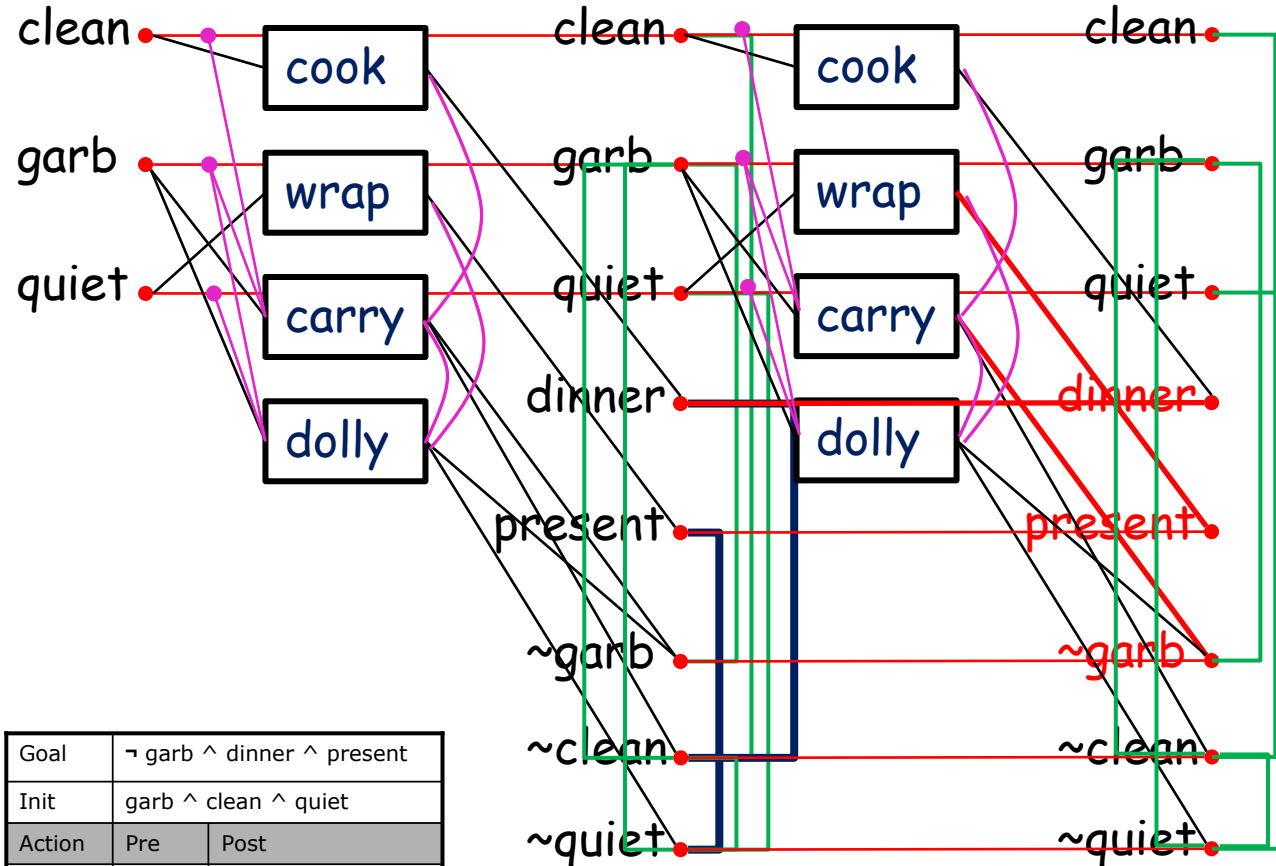
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example



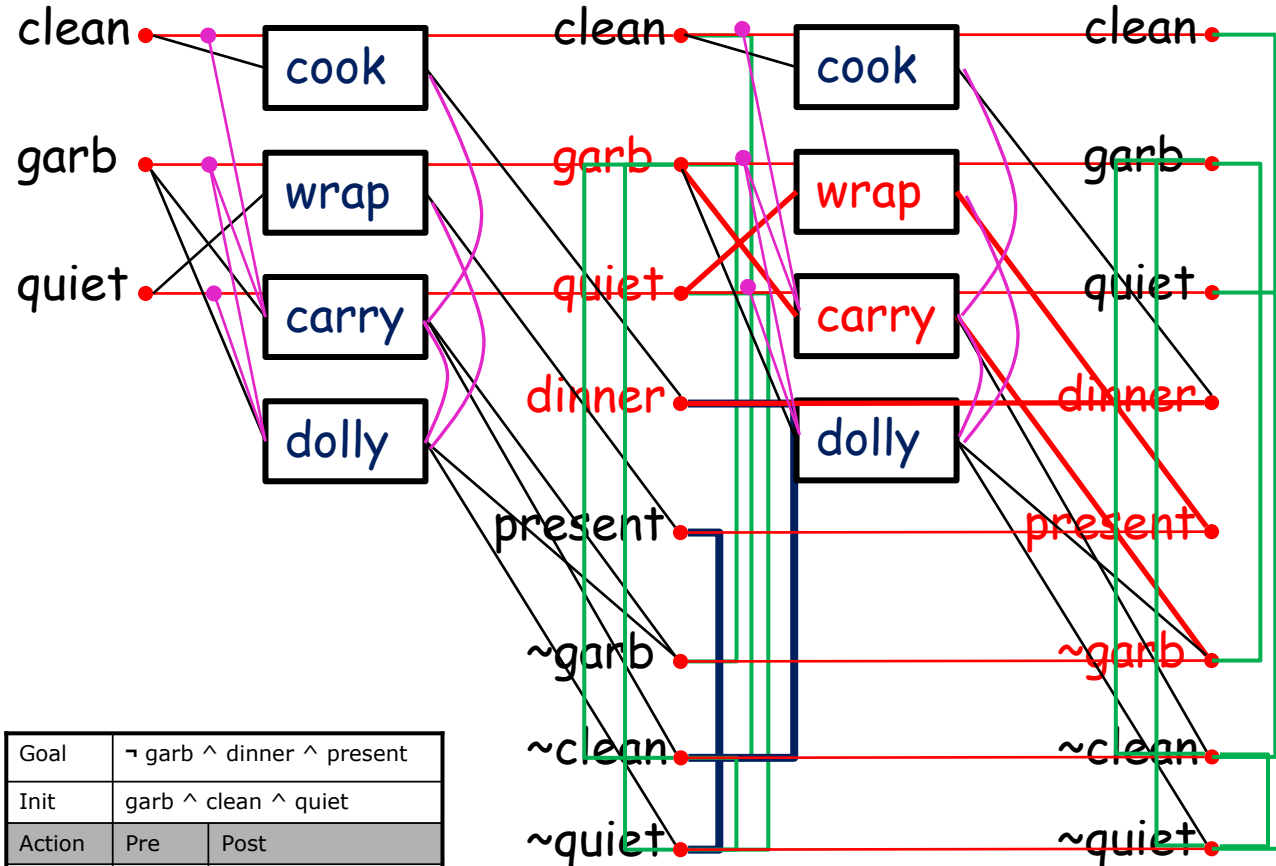
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example



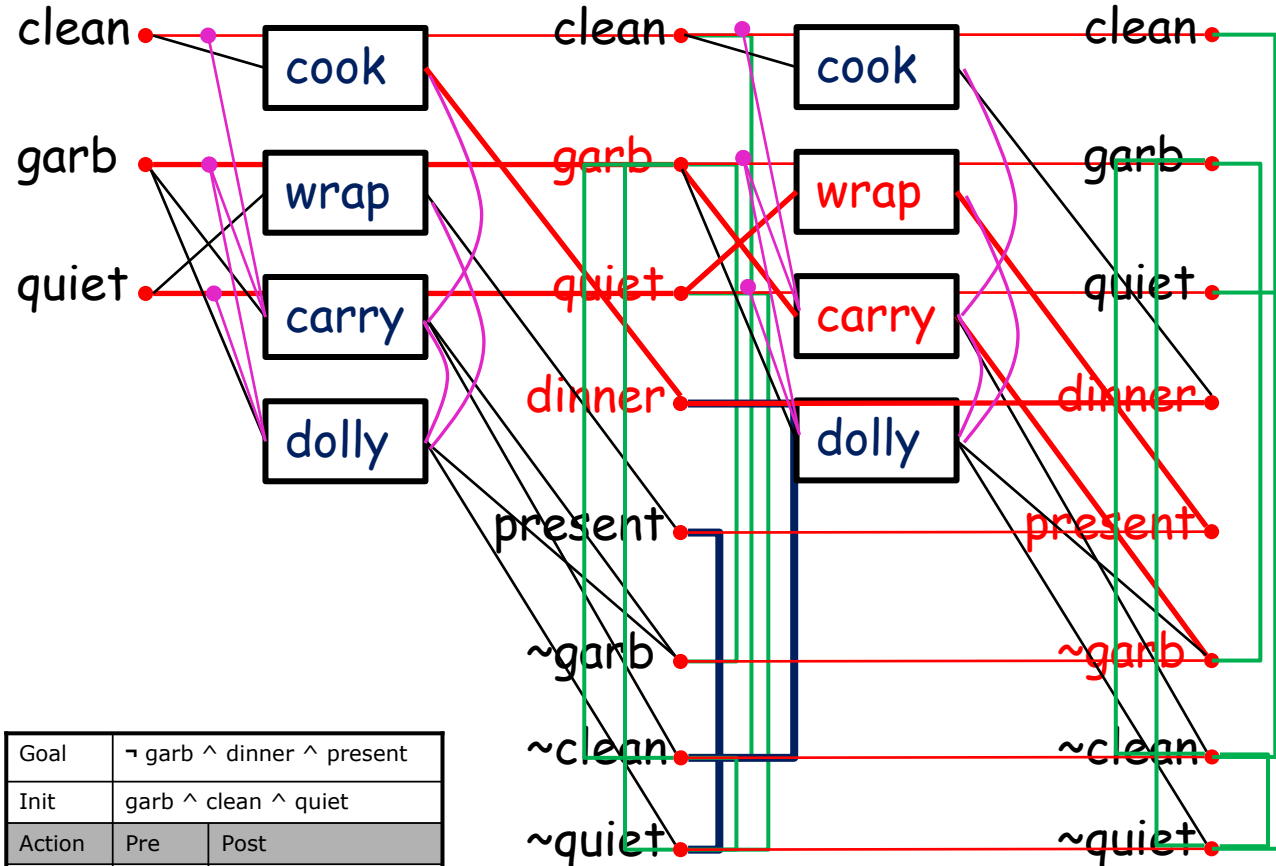
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example



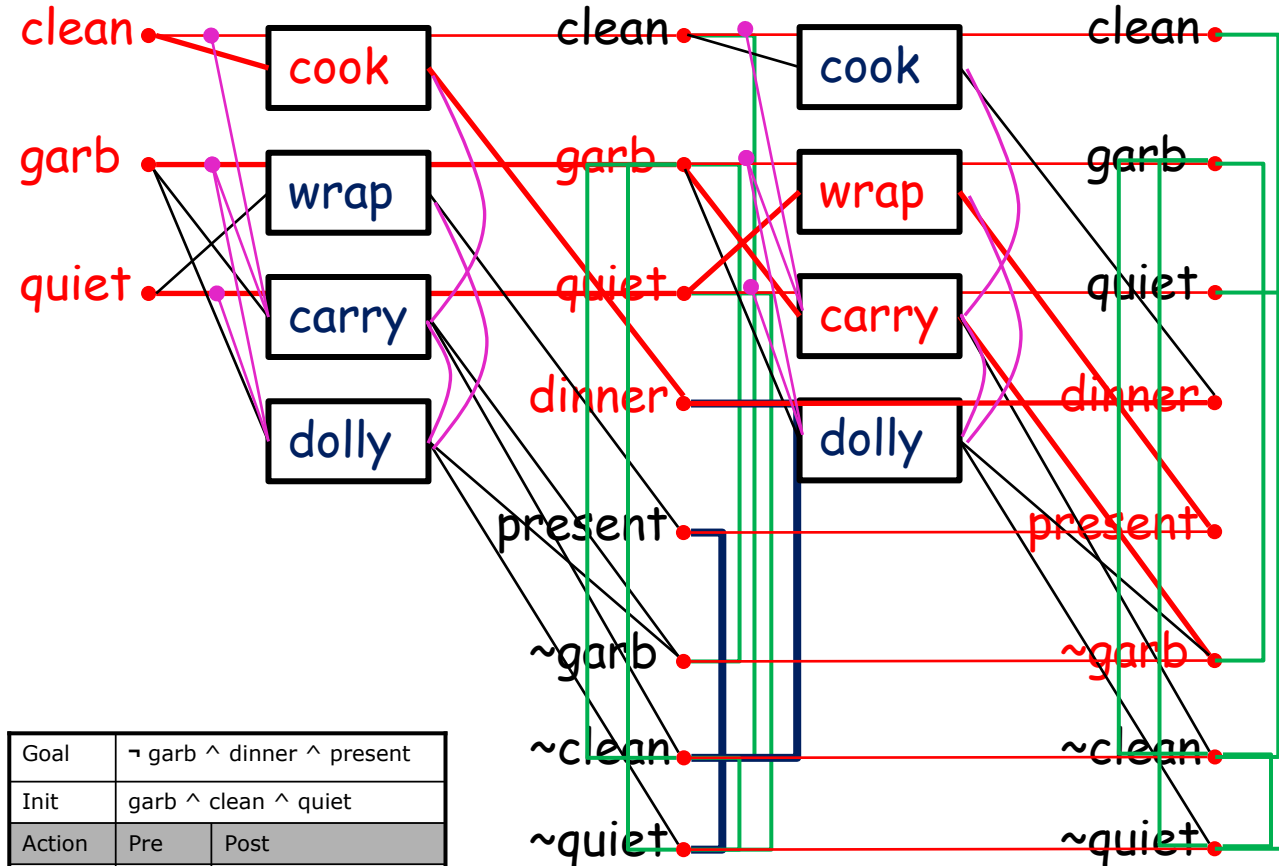
| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example



| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |

# Birthday Dinner Example



| Goal   | $\neg \text{garb} \wedge \text{dinner} \wedge \text{present}$ |                                             |
|--------|---------------------------------------------------------------|---------------------------------------------|
| Init   | $\text{garb} \wedge \text{clean} \wedge \text{quiet}$         |                                             |
| Action | Pre                                                           | Post                                        |
| Cook   | clean                                                         | dinner                                      |
| Wrap   | quiet                                                         | present                                     |
| Carry  | garb                                                          | $\neg \text{garb} \wedge \neg \text{clean}$ |
| Dolly  | garb                                                          | $\neg \text{garb} \wedge \neg \text{quiet}$ |



# Complexity of Planning and Other Approaches

---

- ❑ PlanSAT – Does a plan exist
- ❑ Bounded PlanSAT – Does a plan exist with solution length  $< k$
- ❑ Planning graph – for heuristic computation
  - Section 10.3.1